MySQL 是怎样运行的 精简版

How MySQL Works: Understanding from the Root

后端面试速查笔记

常用命令 · 存储与索引 · 查询优化 · 事务与锁

目录

MySQL 常用命令速查

- 连接与基本操作
- DDL（数据定义）
- DML（数据操作）
- 查询
- 事务
- 锁
- EXPLAIN 与性能分析
- 用户与权限
- 备份与恢复
- 常用系统变量与状态

InnoDB 记录结构

- InnoDB 页简介
- COMPACT 行格式
- 行溢出
- Dynamic 与 Compressed 行格式

InnoDB 数据页结构

- 数据页的七个组成部分
- 记录在页中的存储
- Page Directory（页目录）
- Page Header（页面头部）
- File Header（文件头部）
- File Trailer（文件尾部）
- 面试要点速记

B+ 树索引

- 没有索引时的查找
- 索引的本质
- 聚簇索引
- 二级索引（辅助索引）
- 联合索引
- InnoDB 中 B+ 树索引的注意事项
- InnoDB vs MyISAM 索引对比
- 创建和删除索引

B+ 树索引的使用

- 索引的代价
- 索引的使用条件
- 索引失效的场景

- 回表的代价与覆盖索引
- 索引用于排序
- 索引用于分组
- 索引设计原则

Buffer Pool

- 什么是 Buffer Pool
- Buffer Pool 内部结构
- flush 链表
- LRU 链表
- 刷脏页
- 多 Buffer Pool 实例
- 其他信息

单表访问方法

- 访问方法概述
- const
- ref
- ref_or_null
- range
- index
- ALL
- 二级索引 + 回表的执行流程
- Index Merge（索引合并）
- 注意事项

连接的原理

- 连接的本质
- 连接类型
- 连接的执行原理
- 驱动表的选择

查询优化规则

- 条件化简
- 外连接消除
- 子查询优化
- 派生表优化

EXPLAIN 详解

- 基本用法
- 各列详解
- 输出格式

事务简介

- 事务的概念
- ACID 特性

事务的状态
MySQL 中的事务用法
redo 日志
为什么需要 redo 日志
redo 日志格式
Mini-Transaction（mtr）
redo log buffer
redo 日志文件
LSN（Log Sequence Number）
checkpoint
崩溃恢复
undo 日志
为什么需要 undo 日志
事务 ID
INSERT 的 undo 日志
DELETE 的 undo 日志
UPDATE 的 undo 日志
版本链
undo 日志的存储
undo 日志的生命周期
事务隔离级别与 MVCC
并发事务的问题
SQL 标准的四个隔离级别
MVCC 原理
READ COMMITTED 的 MVCC 实现
REPEATABLE READ 的 MVCC 实现
当前读与快照读
幻读的解决
purge 机制
锁
并发控制与锁的分类
表级锁
行级锁（InnoDB）
加锁分析
死锁
实战：常见加锁场景

常用命令速查

MySQL 常用命令速查

连接与基本操作

```
-- 连接数据库
mysql -u root -p
mysql -h 127.0.0.1 -P 3306 -u root -p database_name

-- 查看数据库/表
SHOW DATABASES;
USE database_name;
SHOW TABLES;
DESC table_name;              -- 查看表结构
SHOW CREATE TABLE table_name; -- 查看建表语句
SHOW TABLE STATUS LIKE 'table_name'\G

-- 查看当前连接信息
SELECT USER(), DATABASE(), VERSION();
SHOW PROCESSLIST;             -- 查看所有连接
KILL connection_id;           -- 杀死连接
```

DDL（数据定义）

```sql
-- 数据库
CREATE DATABASE db_name DEFAULT CHARSET utf8mb4;
DROP DATABASE db_name;
ALTER DATABASE db_name DEFAULT CHARSET utf8mb4;

-- 建表
CREATE TABLE users (
    id        BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    name      VARCHAR(64)  NOT NULL DEFAULT '',
    email     VARCHAR(128) NOT NULL,
    status    TINYINT      NOT NULL DEFAULT 1,
    created_at DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_email (email),
    UNIQUE KEY uk_name (name)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 改表
ALTER TABLE users ADD COLUMN age INT NOT NULL DEFAULT 0 AFTER name;
ALTER TABLE users DROP COLUMN age;
ALTER TABLE users MODIFY COLUMN name VARCHAR(128) NOT NULL;
ALTER TABLE users CHANGE COLUMN name username VARCHAR(128) NOT NULL;
ALTER TABLE users RENAME TO members;

-- 索引
CREATE INDEX idx_name ON users(name);
CREATE UNIQUE INDEX uk_email ON users(email);
ALTER TABLE users ADD INDEX idx_status_created(status, created_at);
DROP INDEX idx_name ON users;
SHOW INDEX FROM users;
```

DML（数据操作）

```sql
-- 插入
INSERT INTO users (name, email) VALUES ('alice', 'a@x.com');
INSERT INTO users (name, email) VALUES ('bob', 'b@x.com'), ('carol', 'c@x.com');
INSERT INTO users (name, email) SELECT name, email FROM old_users;
INSERT INTO users (name, email) VALUES ('alice', 'a@x.com')
  ON DUPLICATE KEY UPDATE email = VALUES(email);
REPLACE INTO users (id, name, email) VALUES (1, 'alice', 'new@x.com');

-- 更新
UPDATE users SET status = 0 WHERE id = 1;
UPDATE users SET status = 0 WHERE created_at < '2024-01-01' LIMIT 1000;

-- 删除
DELETE FROM users WHERE id = 1;
DELETE FROM users WHERE status = 0 LIMIT 1000;
TRUNCATE TABLE users;  -- 清空表（DDL 操作，不可回滚）
```

查询

```sql
-- 基本查询
SELECT * FROM users WHERE id = 1;
SELECT name, email FROM users WHERE status = 1 ORDER BY created_at DESC LIMIT 10;
SELECT status, COUNT(*) AS cnt FROM users GROUP BY status HAVING cnt > 10;

-- 连接
SELECT u.name, o.amount
FROM users u
INNER JOIN orders o ON u.id = o.user_id
WHERE o.created_at > '2024-01-01';

SELECT u.name, o.amount
FROM users u
LEFT JOIN orders o ON u.id = o.user_id;

-- 子查询
SELECT * FROM users WHERE id IN (SELECT user_id FROM orders WHERE amount > 100);
SELECT * FROM users WHERE EXISTS (SELECT 1 FROM orders WHERE orders.user_id = users.id);

-- 分页
SELECT * FROM users ORDER BY id LIMIT 10 OFFSET 20;        -- 偏移量分页（大偏移慢）
SELECT * FROM users WHERE id > 1000 ORDER BY id LIMIT 10;   -- 游标分页（推荐）

-- UNION
SELECT name FROM users UNION ALL SELECT name FROM admins;   -- 保留重复
SELECT name FROM users UNION SELECT name FROM admins;       -- 去重
```

事务

```sql
-- 显式事务
BEGIN;                        -- 或 START TRANSACTION
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;

-- 回滚
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
ROLLBACK;

-- 保存点
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
SAVEPOINT sp1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
ROLLBACK TO sp1;  -- 只回滚到 sp1，id=1 的修改保留
COMMIT;

-- 隔离级别
SELECT @@transaction_isolation;                             -- 查看当前隔离级别
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;    -- MySQL 默认
```

锁

```sql
-- 锁定读
SELECT * FROM users WHERE id = 1 FOR UPDATE;            -- 加 X 锁
SELECT * FROM users WHERE id = 1 FOR SHARE;             -- 加 S 锁（8.0+）
SELECT * FROM users WHERE id = 1 LOCK IN SHARE MODE;    -- 加 S 锁（兼容写法）

-- 查看锁等待
SELECT * FROM information_schema.INNODB_LOCKS;
SELECT * FROM information_schema.INNODB_LOCK_WAITS;
-- 8.0+
SELECT * FROM performance_schema.data_locks;
SELECT * FROM performance_schema.data_lock_waits;

-- 死锁信息
SHOW ENGINE INNODB STATUS\G

-- 表锁
LOCK TABLES users READ;
LOCK TABLES users WRITE;
UNLOCK TABLES;
```

EXPLAIN 与性能分析

```sql
-- EXPLAIN
EXPLAIN SELECT * FROM users WHERE name = 'alice';
EXPLAIN FORMAT=JSON SELECT * FROM users WHERE name = 'alice';

-- 慢查询
SHOW VARIABLES LIKE 'slow_query%';
SET GLOBAL slow_query_log = ON;
SET GLOBAL long_query_time = 1;

-- 查看状态
SHOW STATUS LIKE 'Handler%';       -- 行操作统计
SHOW STATUS LIKE 'Innodb_rows%';   -- InnoDB 行操作统计
SHOW VARIABLES LIKE 'innodb%';     -- InnoDB 配置

-- profiling
SET profiling = 1;
SELECT * FROM users WHERE id = 1;
SHOW PROFILES;
SHOW PROFILE FOR QUERY 1;
```

用户与权限

```sql
-- 创建用户
CREATE USER 'app'@'%' IDENTIFIED BY 'password';
CREATE USER 'app'@'192.168.1.%' IDENTIFIED BY 'password';

-- 授权
GRANT SELECT, INSERT, UPDATE ON db_name.* TO 'app'@'%';
GRANT ALL PRIVILEGES ON db_name.* TO 'app'@'%';
FLUSH PRIVILEGES;

-- 查看权限
SHOW GRANTS FOR 'app'@'%';

-- 撤销权限
REVOKE INSERT ON db_name.* FROM 'app'@'%';

-- 修改密码
ALTER USER 'app'@'%' IDENTIFIED BY 'new_password';

-- 删除用户
DROP USER 'app'@'%';
```

备份与恢复

```bash
-- mysqldump（逻辑备份）
mysqldump -u root -p db_name > backup.sql
mysqldump -u root -p --all-databases > all_backup.sql
mysqldump -u root -p db_name table1 table2 > tables_backup.sql
mysqldump -u root -p --single-transaction db_name > backup.sql  -- InnoDB 一致性备份

-- 恢复
mysql -u root -p db_name < backup.sql
```

常用系统变量与状态

变量	说明	查看命令
`innodb_buffer_pool_size`	Buffer Pool 大小	`SHOW VARIABLES LIKE 'innodb_buffer_pool_size'`
`innodb_flush_log_at_trx_commit`	redo 刷盘策略（0/1/2）	`SHOW VARIABLES LIKE 'innodb_flush%'`
`innodb_log_file_size`	redo 日志文件大小	`SHOW VARIABLES LIKE 'innodb_log%'`
`max_connections`	最大连接数	`SHOW VARIABLES LIKE 'max_connections'`
`innodb_lock_wait_timeout`	锁等待超时（秒）	`SHOW VARIABLES LIKE 'innodb_lock%'`
`transaction_isolation`	事务隔离级别	`SELECT @@transaction_isolation`

autocommit	自动提交	SELECT @@autocommit
innodb_autoinc_lock_mode	自增锁模式（0/1/2）	SHOW VARIABLES LIKE 'innodb_autoinc%'

第一部分：存储引擎与索引

InnoDB 记录结构

InnoDB 页简介

InnoDB 以页（Page）为单位管理存储空间，页是磁盘与内存交互的基本单位，默认大小 16KB。读取数据时最少从磁盘读 16KB 到内存，刷盘时也至少写 16KB。

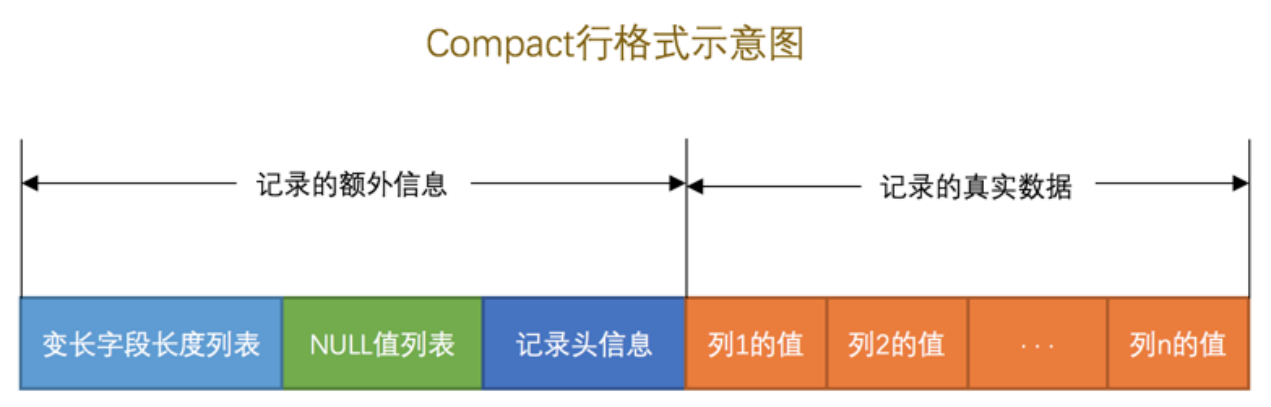
COMPACT 行格式

InnoDB 支持 4 种行格式：`Compact`、`Redundant`、`Dynamic`、`Compressed`。MySQL 5.7+ 默认使用 `Dynamic`，它与 `Compact` 结构基本一致，仅行溢出处理不同。

指定行格式：

```
CREATE TABLE t (...) ROW_FORMAT=COMPACT;
ALTER TABLE t ROW_FORMAT=DYNAMIC;
```

一条 COMPACT 格式的记录由额外信息和真实数据两部分组成：



COMPACT行格式

记录的额外信息

额外信息包含三部分：变长字段长度列表、NULL 值列表、记录头信息。

变长字段长度列表

`VARCHAR(M)`、`VARBINARY(M)`、`TEXT`、`BLOB` 等变长类型的列，需要在记录开头存储其实际数据占用的字节数。各变长字段的长度按列顺序逆序排列，形成变长字段长度列表。

长度字段占用 1 字节还是 2 字节，规则如下（`W` 为字符集单字符最大字节数，`M` 为声明的字符数，`L` 为实际字节数）：

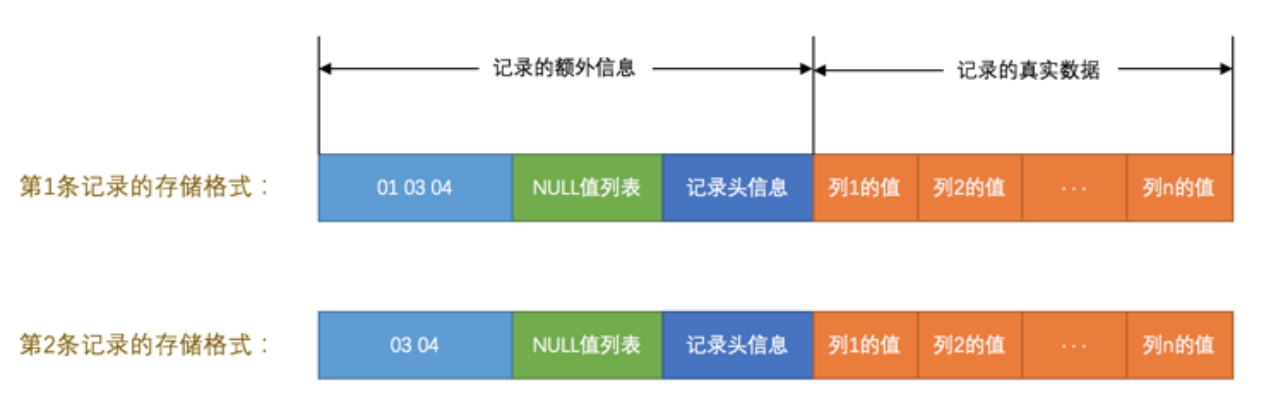
条件	长度字段占用
`M × W ≤ 255`	1 字节
`M × W > 255` 且 `L ≤ 127`	1 字节

`M × W > 255` 且 `L > 127`	2 字节

区分 1/2 字节的方法：用该字节的最高位作标志位，0 表示单独一个字段长度，1 表示这是半个字段长度（即用了 2 字节）。

关键细节：

- 值为 NULL 的列不存储长度，NULL 值由 NULL 值列表处理。
- 若表中所有列都是定长类型，则不存在变长字段长度列表。
- `CHAR(M)` 在变长字符集（如 `utf8`）下也会被加入变长字段长度列表；在定长字符集（如 `ascii`）下不会。



变长字段长度列表示例

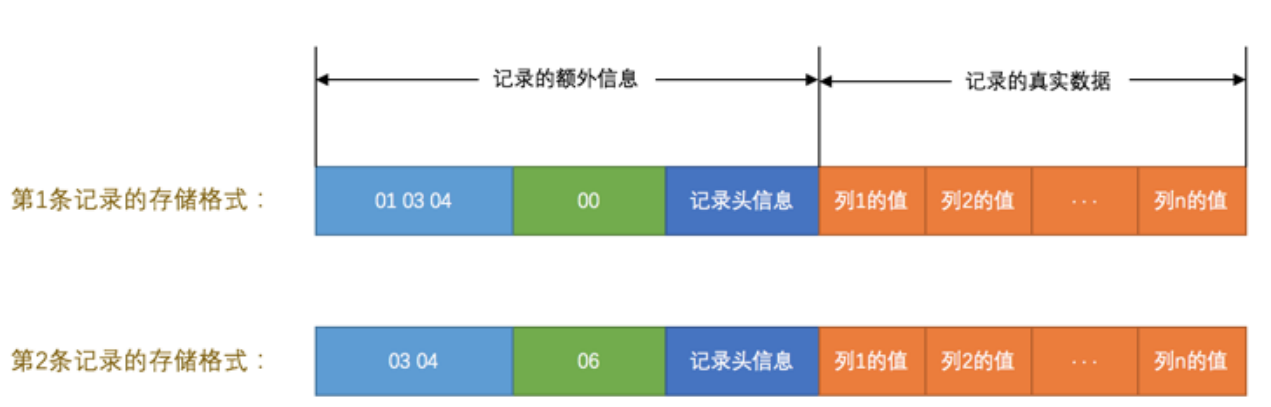
NULL 值列表

将允许为 NULL 的列（排除主键列和 `NOT NULL` 列）按列顺序逆序，每列对应一个二进制位：

- `1` → 该列值为 NULL
- `0` → 该列值不为 NULL

位数不足整字节时，高位补 0。例如表有 3 个可 NULL 列，则 NULL 值列表占 1 字节；9 个可 NULL 列则占 2 字节。

若表中没有允许 NULL 的列，则 NULL 值列表不存在。

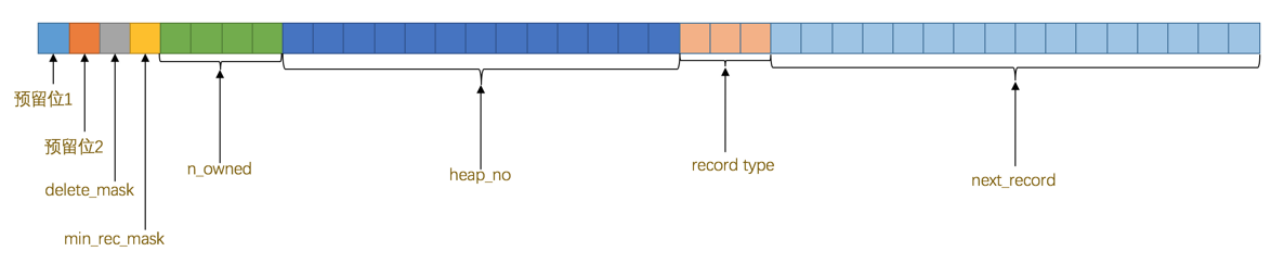


NULL值列表示例

记录头信息

固定占用 5 字节（40 bit），各字段含义：

名称	大小(bit)	描述
1	1	未使用
2	1	未使用
delete_mask	1	标记记录是否被删除
min_rec_mask	1	B+ 树每层非叶子节点的最小记录标记
n_owned	4	当前记录拥有的记录数（页目录相关）
heap_no	13	当前记录在记录堆中的位置
record_type	3	0=普通记录, 1=B+树非叶节点, 2=最小记录, 3=最大记录
next_record	16	下一条记录的相对位置（形成单链表）



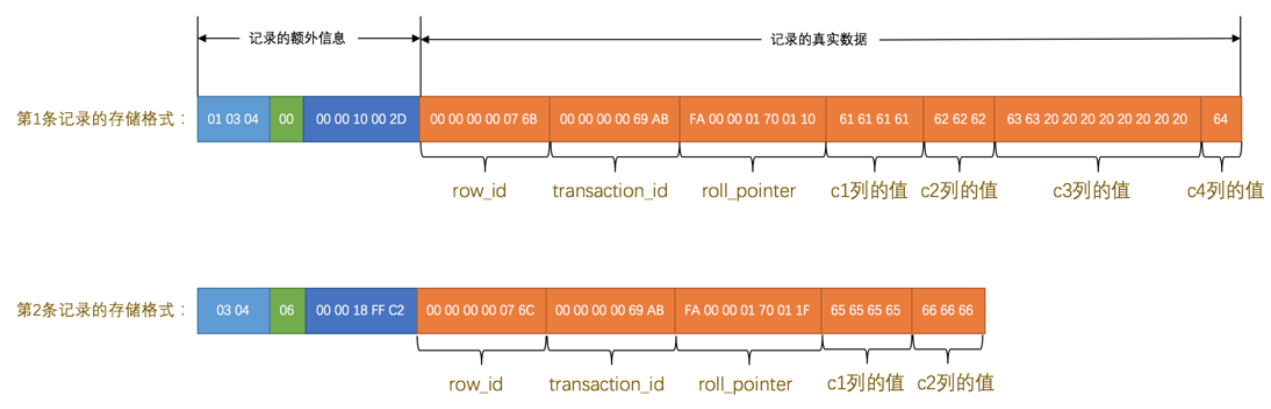
记录头信息

记录的真实数据

真实数据部分存储用户定义的各列数据。

`CHAR(M)` 在定长字符集下固定占用 `M` 字节，不足部分用空格（`0x20`）填充。在变长字符集下至少占用 `M` 字节（避免原地更新时产生碎片），最多占用 `M × W` 字节。

值为 NULL 的列不在真实数据区占用空间，已通过 NULL 值列表标记。



真实数据示例

隐藏列

InnoDB 为每条记录自动添加隐藏列：

列名	是否必须	大小	描述
`DB_ROW_ID`	否	6 字节	行 ID，无主键且无唯一索引时自动生成作为隐式主键
`DB_TRX_ID`	是	6 字节	事务 ID，标识最后修改该记录的事务
`DB_ROLL_PTR`	是	7 字节	回滚指针，指向 undo log 中该记录的上一个版本

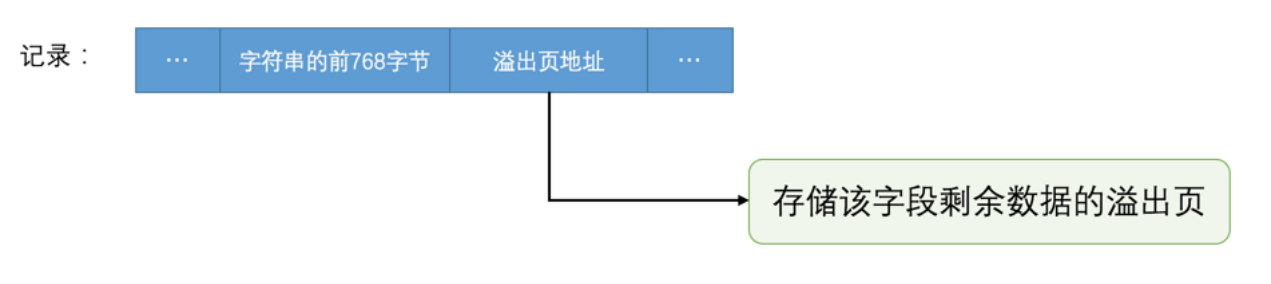
主键生成策略：用户定义主键 → `UNIQUE` 键 → 自动添加 `DB_ROW_ID`。`DB_TRX_ID` 和 `DB_ROLL_PTR` 始终存在，是 MVCC 机制的基础。

行溢出

一个页 16KB（16384 字节），而 `VARCHAR(M)` 最多能存 65532 字节（加上 2 字节长度 + 1 字节 NULL 标识 = 65535 上限）。当单条记录数据量超过页容量时，就会发生行溢出。

MySQL 规定一个页中至少存放两行记录。对于单列表，行溢出的临界点约为 8098 字节（由公式 `136 + 2 × (27 + n) > 16384` 推导）。多列表临界点更低。

COMPACT/Redundant 格式的溢出处理：在本页存储前 768 字节 + 20 字节的溢出页地址指针，剩余数据存入溢出页。



行溢出示意

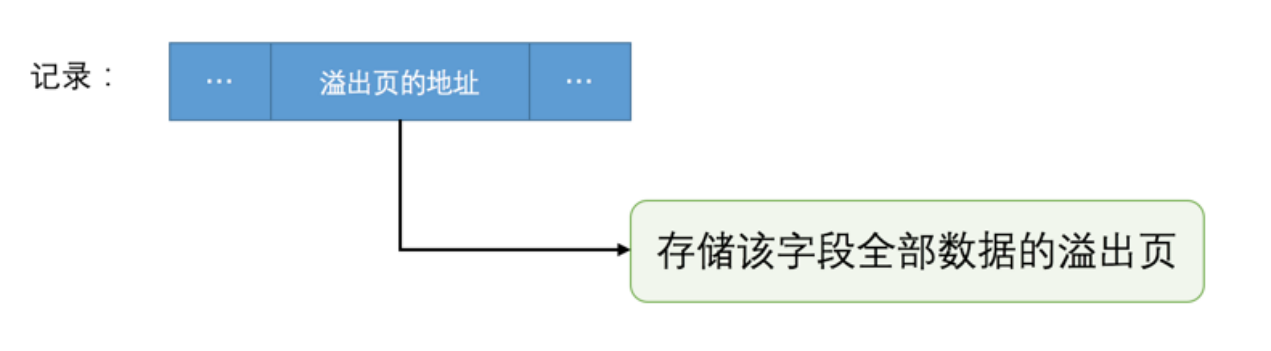
不只是 `VARCHAR`，`TEXT` 和 `BLOB` 类型数据量过大时同样会行溢出。

Dynamic 与 Compressed 行格式

`Dynamic`（MySQL 5.7+ 默认）和 `Compressed` 与 `Compact` 结构基本一致，区别在于行溢出处理方式：

行格式	溢出处理
Compact / Redundant	本页保留前 768 字节 + 20 字节溢出页指针
Dynamic / Compressed	本页仅存 20 字节溢出页指针，所有数据存入溢出页

`Compressed` 额外会对页面数据进行压缩以节省磁盘空间。



Dynamic行溢出处理

InnoDB 数据页结构

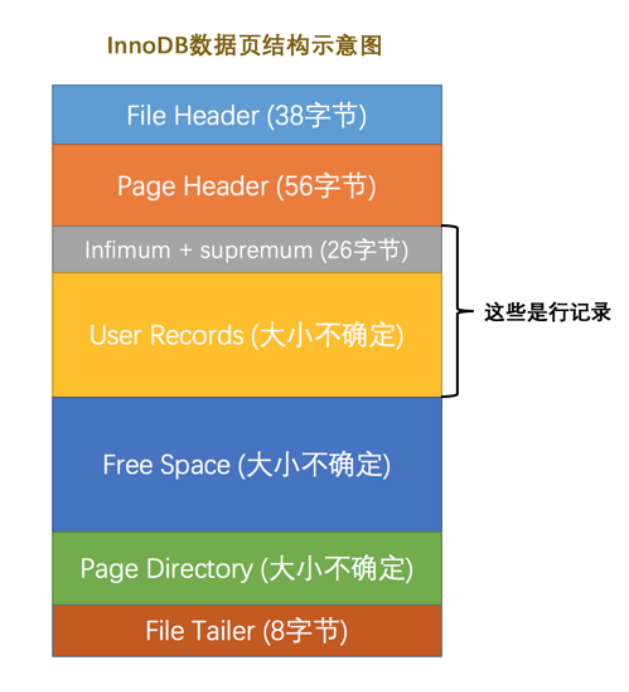
InnoDB 以页为单位管理存储空间，默认页大小 16KB。存放表中行记录的页称为数据页（类型 FIL_PAGE_INDEX，即索引页）。

数据页的七个组成部分

一个数据页的 16KB 空间划分为 7 个部分：

名称	大小	说明
File Header	38 字节	页的通用信息：页号、校验和、前后页指针等
Page Header	56 字节	数据页专有状态：记录数、槽数、空闲空间位置等
Infimum + Supremum	26 字节	两条虚拟记录，分别代表页内最小和最大记录
User Records	不固定	实际存储的行记录
Free Space	不固定	尚未使用的空间
Page Directory	不固定	由若干槽（Slot）组成，存储分组中最大记录的地址偏移量
File Trailer	8 字节	校验和 + LSN，用于检测页写入是否完整

插入记录时，从 Free Space 中划出一块空间分配给 User Records。当 Free Space 耗尽，需要申请新的数据页。



记录在页中的存储

记录头信息

每条记录都有 5 字节的记录头信息，核心字段：

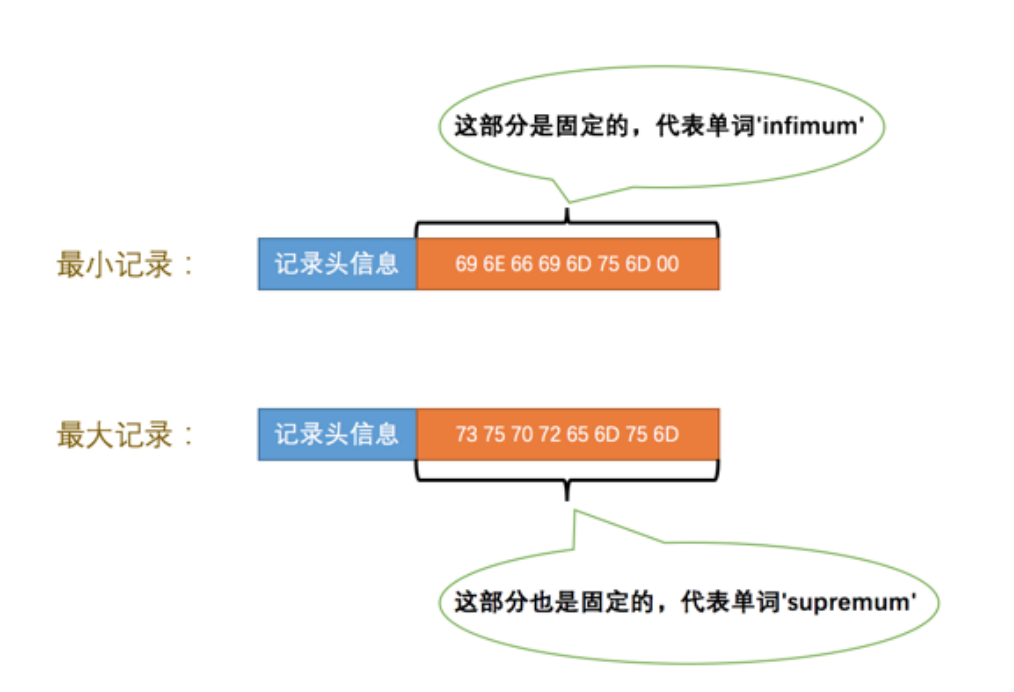
字段	位数	说明
`delete_mask`	1	删除标记。1 表示已删除，记录仍留在页中，空间可复用
`min_rec_mask`	1	B+ 树每层非叶子节点的最小记录会标记为 1
`n_owned`	4	该记录所在分组的记录数（仅组内最大记录维护此值）
`heap_no`	13	记录在本页中的堆位置编号
`record_type`	3	0=普通记录，1=B+ 树非叶节点记录，2=最小记录，3=最大记录
`next_record`	16	指向下一条记录真实数据的地址偏移量

Infimum 和 Supremum 记录

每个数据页自动包含两条虚拟记录：

- Infimum（最小记录）：`heap_no = 0`，`record_type = 2`
- Supremum（最大记录）：`heap_no = 1`，`record_type = 3`

这两条记录存放在 `Infimum + Supremum` 区域，不在 `User Records` 中。它们各由 5 字节记录头信息 + 8 字节固定数据组成。无论插入多少用户记录，Infimum 始终最小，Supremum 始终最大。



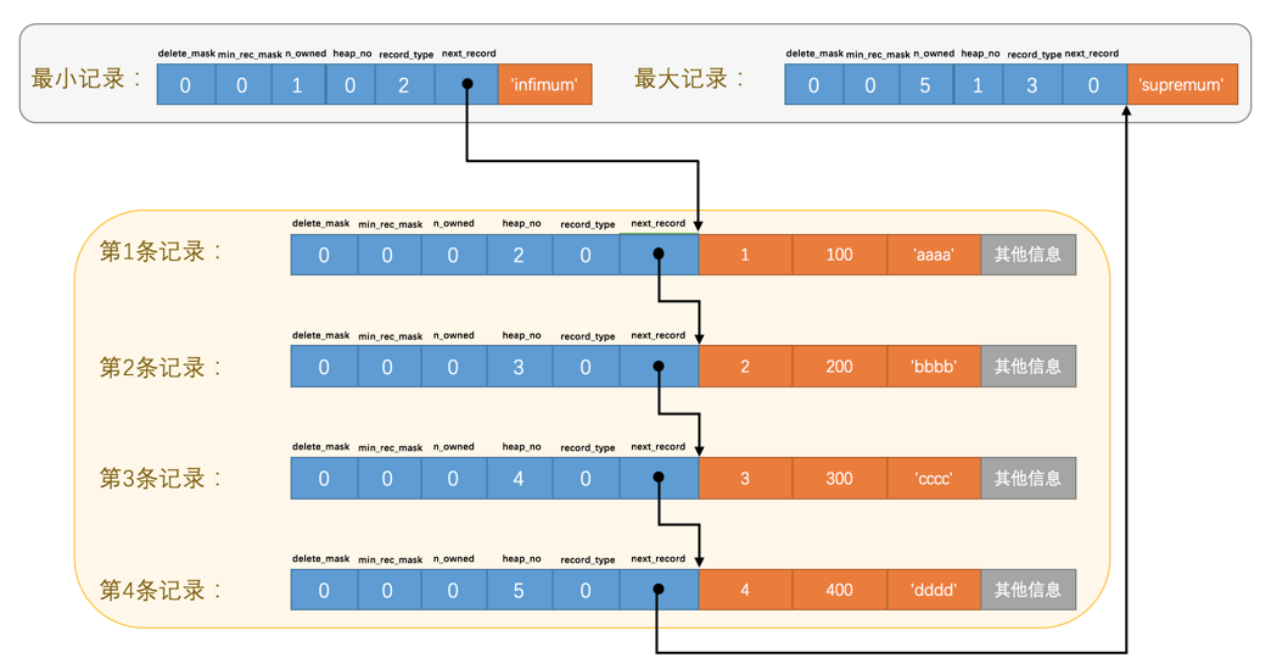
记录的排列（next_record 单链表）

页中所有记录通过 `next_record` 按主键值从小到大串成单链表：

```
Infimum → 主键最小的用户记录 → ... → 主键最大的用户记录 → Supremum
```

关键规则：

- `next_record` 是从当前记录真实数据到下一条记录真实数据的偏移量（向左读头信息，向右读数据）
- 链表顺序与插入顺序无关，始终按主键排序
- Supremum 的 `next_record = 0`，表示链表终止

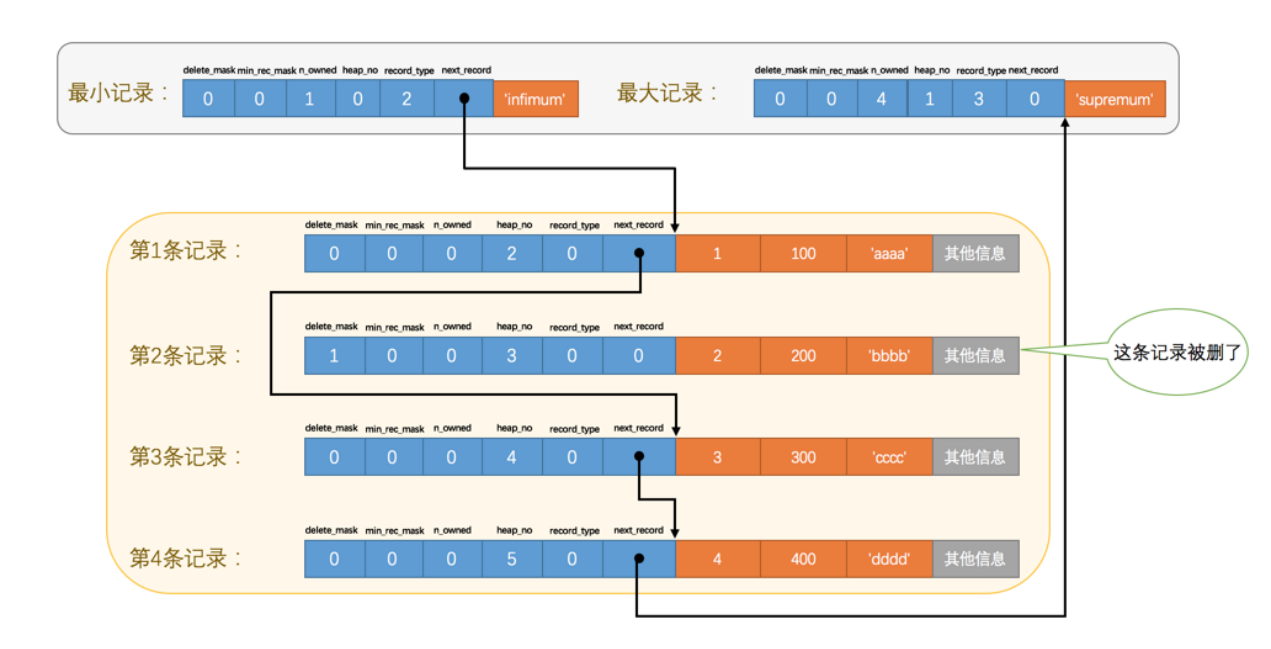


删除记录的处理

删除一条记录时：

1. 将该记录的 `delete_mask` 置为 1，不回收存储空间
2. 该记录的 `next_record` 置为 0
3. 前一条记录的 `next_record` 指向后一条记录，链表保持连通
4. 所在组的 `n_owned` 值减 1

被删除的记录通过 `next_record` 组成垃圾链表，其空间可被后续新记录复用。



Page Directory（页目录）

单链表只能顺序遍历，记录多时效率低。Page Directory 通过分组 + 二分查找加速检索。

分组规则

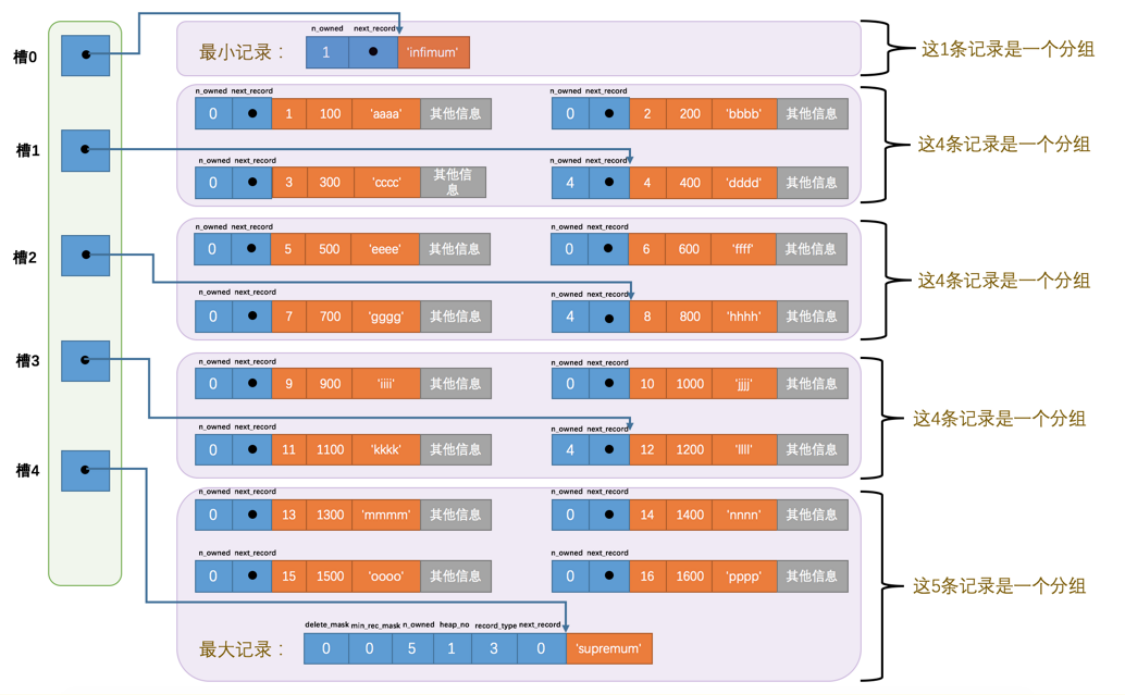
1. 将所有未删除的记录（含 Infimum 和 Supremum）分成若干组
2. 每组的最后一条记录（组内主键最大者）的 `n_owned` 记录该组的记录条数
3. 将每组最后一条记录的地址偏移量提取出来，按序存入 `Page Directory`，每个偏移量称为一个槽（Slot）

各组记录数的约束：

分组	允许的记录条数
Infimum 所在组	恰好 1 条
Supremum 所在组	1 ~ 8 条
其余中间组	4 ~ 8 条

分组的动态维护过程：

1. 初始状态只有 Infimum 和 Supremum 两条记录，分属两个组
2. 每插入一条记录，找到主键值刚好大于它的槽，将该槽对应记录的 `n_owned` 加 1
3. 当某组记录数达到 8 条后再插入时，拆分为 4 + 5 两组，Page Directory 新增一个槽



查找过程（二分查找）

在数据页中按主键查找记录分两步：

第一步：二分法定位槽

槽从 0 到 n 编号，对应的主键值从小到大排列。设 `low = 0`，`high = n`：

1. 取中间槽 `mid = (low + high) / 2`
2. 比较 `mid` 槽对应记录的主键值与目标值
3. 目标值大，则 `low = mid`；目标值小，则 `high = mid`
4. 当 `high - low = 1` 时，目标记录在 `high` 槽对应的组中

第二步：链表遍历组内记录

通过 `low` 槽对应记录的 `next_record` 找到 `high` 槽所在组的第一条记录，沿链表顺序遍历（最多 8 次），找到目标记录。

示例：查找主键值为 6 的记录，假设有 5 个槽（0~4）：

```
1. mid = (0+4)/2 = 2, 槽2主键=8 > 6 → high=2
2. mid = (0+2)/2 = 1, 槽1主键=4 < 6 → low=1
3. high - low = 1, 目标在槽2对应的组中
4. 取槽1记录(主键4)的 next_record → 主键5 → 主键6, 找到
```

Page Header（页面头部）

56 字节，存储数据页的状态信息：

字段	大小	说明
PAGE_N_DIR_SLOTS	2 字节	Page Directory 中的槽数量
PAGE_HEAP_TOP	2 字节	Free Space 的起始地址
PAGE_N_HEAP	2 字节	页中所有记录数（含 Infimum/Supremum 和已删除记录）
PAGE_FREE	2 字节	垃圾链表的头节点地址
PAGE_GARBAGE	2 字节	已删除记录占用的总字节数
PAGE_LAST_INSERT	2 字节	最后一次插入记录的位置
PAGE_DIRECTION	2 字节	最后一次插入的方向（左/右，即主键比上一条小/大）
PAGE_N_DIRECTION	2 字节	同一方向连续插入的记录数
PAGE_N_RECS	2 字节	有效用户记录数（不含 Infimum/Supremum 和已删除记录）
PAGE_MAX_TRX_ID	8 字节	修改该页的最大事务 ID（仅二级索引页有意义）
PAGE_LEVEL	2 字节	该页在 B+ 树中的层级（叶子节点为 0）
PAGE_INDEX_ID	8 字节	该页所属索引的 ID
PAGE_BTR_SEG_LEAF	10 字节	B+ 树叶子段头信息（仅 Root 页）
PAGE_BTR_SEG_TOP	10 字节	B+ 树非叶子段头信息（仅 Root 页）

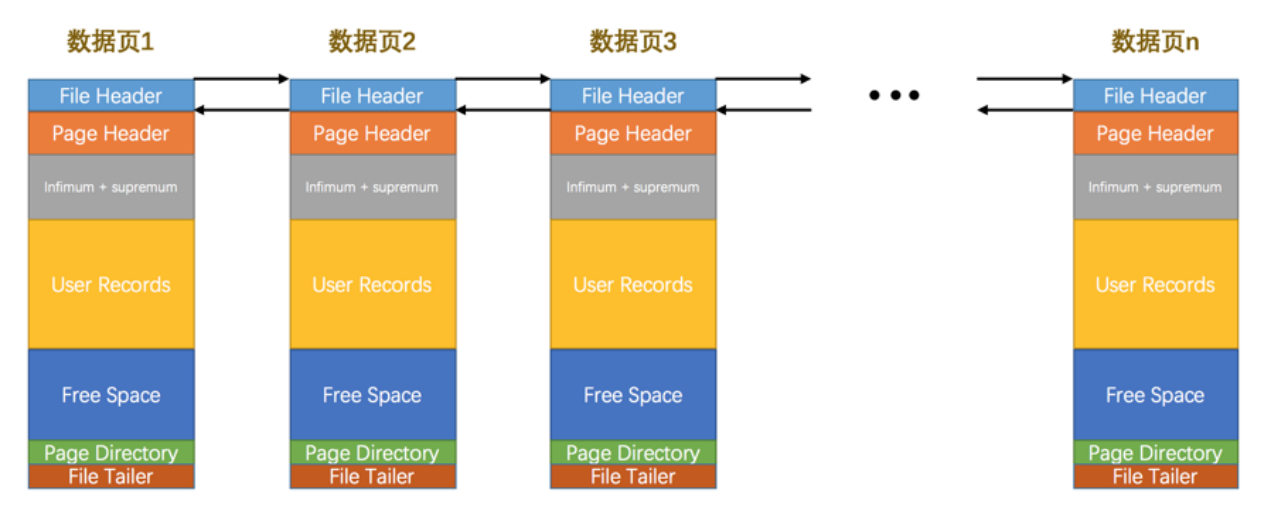
File Header（文件头部）

38 字节，所有类型的页通用：

字段	大小	说明
FIL_PAGE_SPACE_OR_CHKSUM	4 字节	页的校验和
FIL_PAGE_OFFSET	4 字节	页号，唯一标识一个页
FIL_PAGE_PREV	4 字节	上一个页的页号
FIL_PAGE_NEXT	4 字节	下一个页的页号
FIL_PAGE_LSN	8 字节	页面最后修改时的日志序列号（LSN）
FIL_PAGE_TYPE	2 字节	页类型（数据页为 `0x45BF`）
FIL_PAGE_FILE_FLUSH_LSN	8 字节	仅系统表空间使用，文件至少刷新到的 LSN

FIL_PAGE_ARCH_LOG_NO_OR_SPACE_ID	4 字节	所属表空间 ID

通过 FIL_PAGE_PREV 和 FIL_PAGE_NEXT，所有数据页组成双向链表，物理上不必连续存储。



File Trailer（文件尾部）

8 字节，用于校验页的写入完整性：

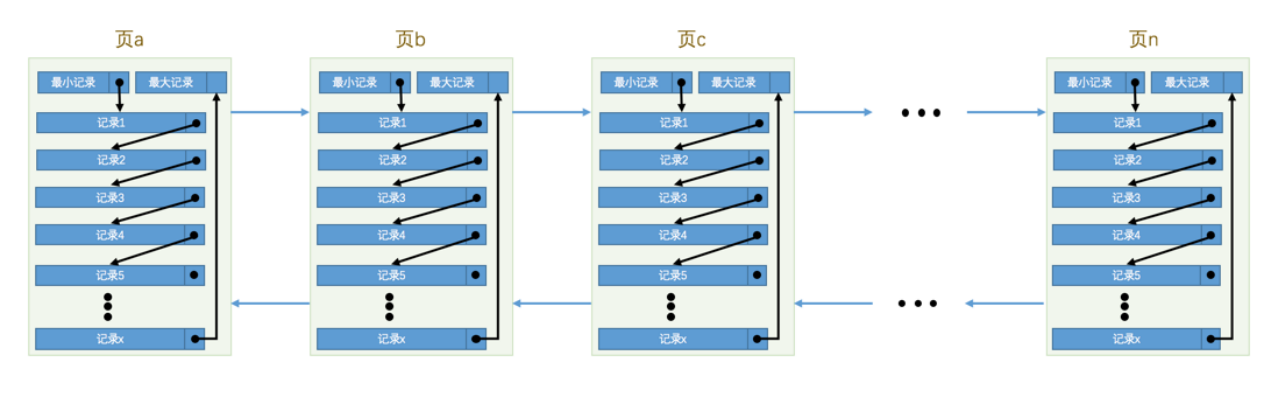
- 前 4 字节：校验和，与 File Header 中的校验和对应。页在内存中修改后先写头部校验和，全部写完后写尾部校验和。若二者不一致，说明写入过程中发生了中断（如断电）。
- 后 4 字节：页面最后修改时的 LSN 低 4 字节，同样用于完整性校验。

面试要点速记

1. 数据页 16KB，分 7 个部分；记录通过 next_record 按主键排序形成单链表
2. 删除记录不回收空间，标记 delete_mask = 1，加入垃圾链表待复用
3. Page Directory 将记录分组，每组 1~8 条，每个槽指向组内最大记录
4. 页内按主键查找：先二分查找定位槽，再遍历组内链表（最多 8 次比较）
5. 数据页之间通过 FIL_PAGE_PREV / NEXT 组成双向链表
6. File Trailer 与 File Header 的校验和配合检测页写入完整性

B+ 树索引

InnoDB 的数据页通过双向链表串联，页内记录按主键组成单向链表，每页有页目录（Page Directory）支持二分查找。索引就是在此基础上构建的多级目录结构。



页与记录的关系

没有索引时的查找

页内查找分两种情况：

- 主键搜索：利用页目录二分定位槽，再遍历槽内分组，速度快。
- 非主键搜索：没有对应的页目录，只能从最小记录开始逐条遍历单链表，效率极低。

跨页查找需要两步：先定位记录所在页，再在页内查找。没有索引时，无论按什么列搜索，都只能沿双向链表从第一页逐页扫描——这就是全表扫描，数据量大时极慢。

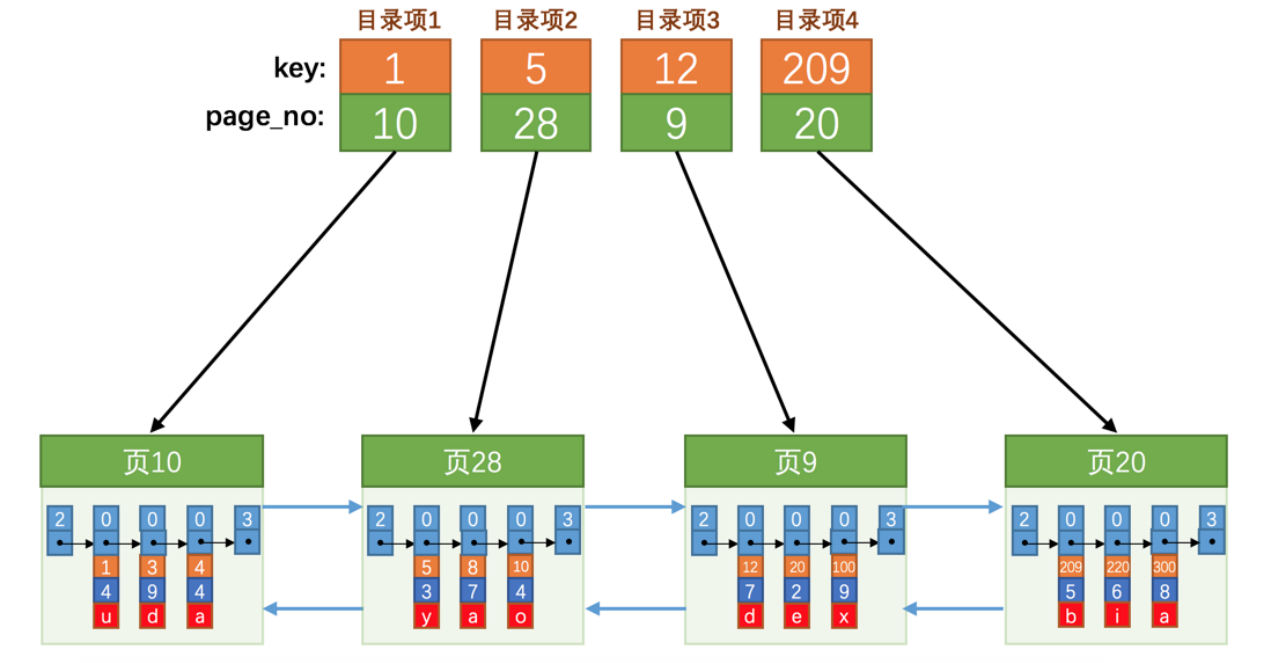
索引的本质

页内目录 → 页间目录

每个数据页内部已有页目录，可以二分查主键。问题在于：页与页之间没有这样的快速定位机制。

为解决跨页查找，需要为所有数据页建立目录项，每个目录项包含：

- `key`：该页中用户记录的最小主键值
- `page_no`：页号



简易目录示意

有了目录，查主键 20 的记录只需：先在目录项中二分定位到目标页（12 < 20 < 209 → 页 9），再在页 9 内二分查找。这就是索引的雏形。

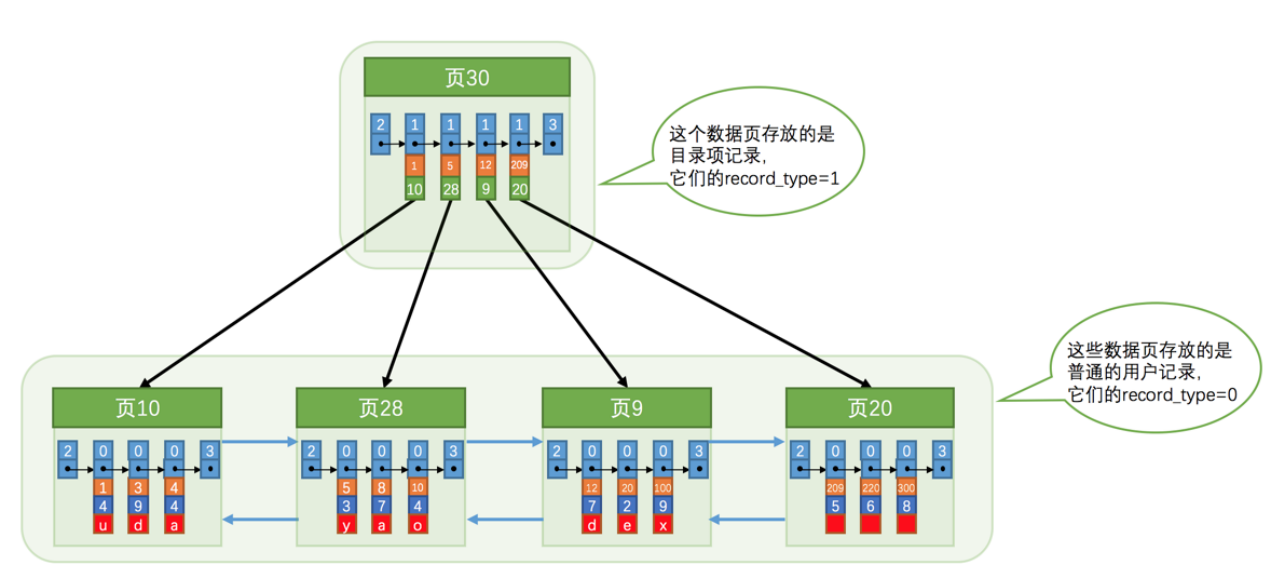
从目录到 B+ 树

上述方案要求目录项在物理上连续存储，存在两个问题：

1. 记录数量增长后，目录项需要的连续空间可能超过单个 16KB 页。
2. 删除页后需要移动后续所有目录项，代价高。

InnoDB 的解决方式：用数据页来存储目录项。目录项记录和用户记录结构相同，通过 `record_type` 区分：

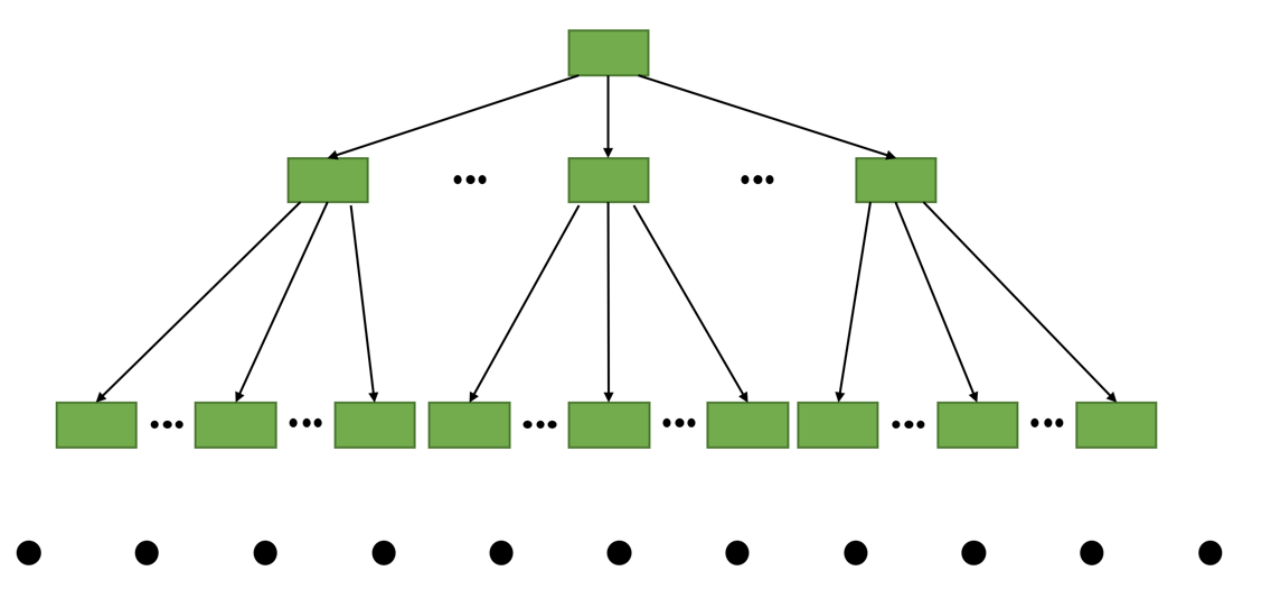
record_type	含义
0	普通用户记录
1	目录项记录
2	最小记录
3	最大记录



目录项记录存储在数据页中

目录项记录只包含主键值 + 页号两个字段，比用户记录小得多，但一个页容量有限。当目录项记录超出单页容量时，就再分配新的目录项页。

多个目录项页之上，再建一层更高级的目录页来索引它们——层层嵌套，最终形成多级树形结构：



B+ 树结构

这就是 B+ 树。

- 叶子节点（最底层，第 0 层）：存放实际用户记录。
- 内节点（非叶子节点）：存放目录项记录。
- 根节点：最顶层的节点。

B+ 树的存储容量

假设叶子节点每页存 100 条用户记录，内节点每页存 1000 条目录项记录：

层数	最大记录数
1	100
2	10 万
3	1 亿
4	1000 亿

实际使用中 B+ 树一般不超过 4 层。查找一条记录最多 4 次页内二分查找，效率极高。

聚簇索引

上面构建的 B+ 树就是聚簇索引（Clustered Index）。它有两个核心特征：

1. 按主键排序组织所有记录和页：
   - 页内记录按主键排成单向链表。
   - 叶子节点（用户记录页）按主键排成双向链表。
   - 同层的内节点页也按主键排成双向链表。
2. 叶子节点存储完整的用户记录（包括所有列和隐藏列）。

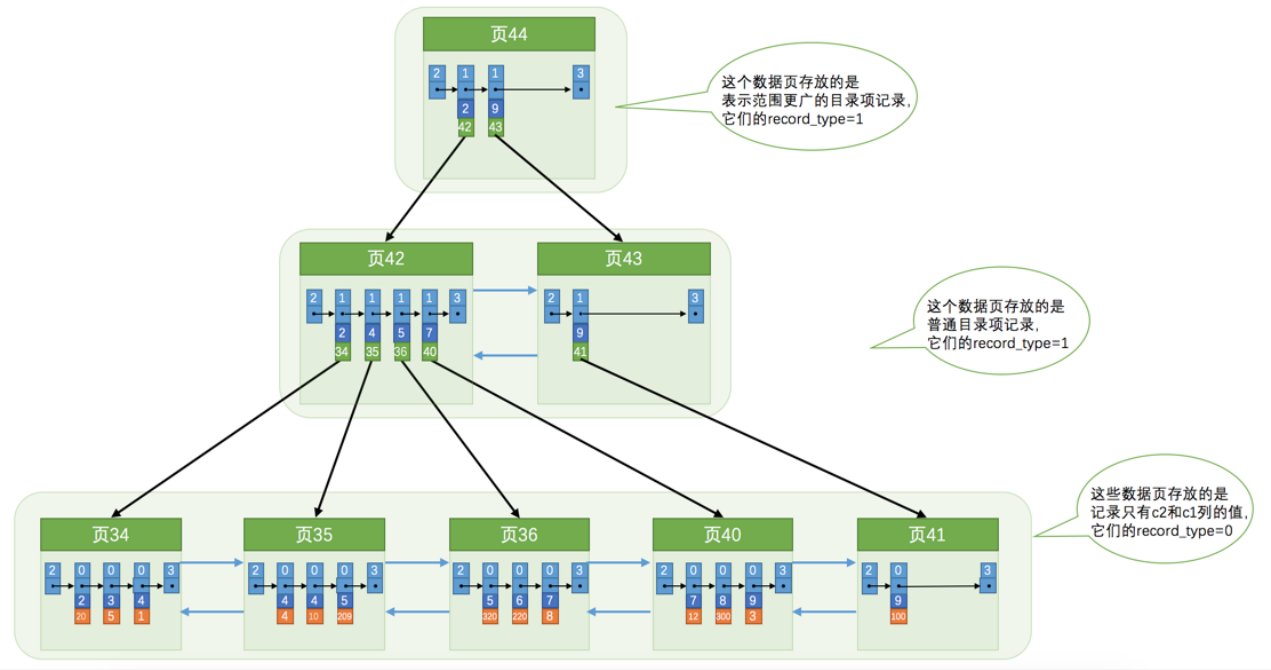
关键要点：

- InnoDB 自动创建聚簇索引，无需手动建。
- 聚簇索引即数据存储方式本身——索引即数据，数据即索引。
- 每张 InnoDB 表有且仅有一个聚簇索引。

二级索引（辅助索引）

聚簇索引只能加速主键查询。若要按其他列查找，需要额外建 B+ 树。

以 `c2` 列建索引为例：



c2 列的二级索引

与聚簇索引的区别：

对比项	聚簇索引	二级索引
排序依据	主键	索引列（如 c2）
叶子节点存储	完整用户记录	索引列值 + 主键值
目录项记录	主键 + 页号	索引列 + 主键 + 页号
查完整记录	一次查找即可	需要回表

回表过程

用 `c2 = 4` 查找完整记录的步骤：

1. 在二级索引 B+ 树中，从根节点逐层定位到叶子节点。
2. 在叶子节点找到匹配记录，取出其中存储的主键值。
3. 拿主键值回到聚簇索引中再查一次，取出完整用户记录。

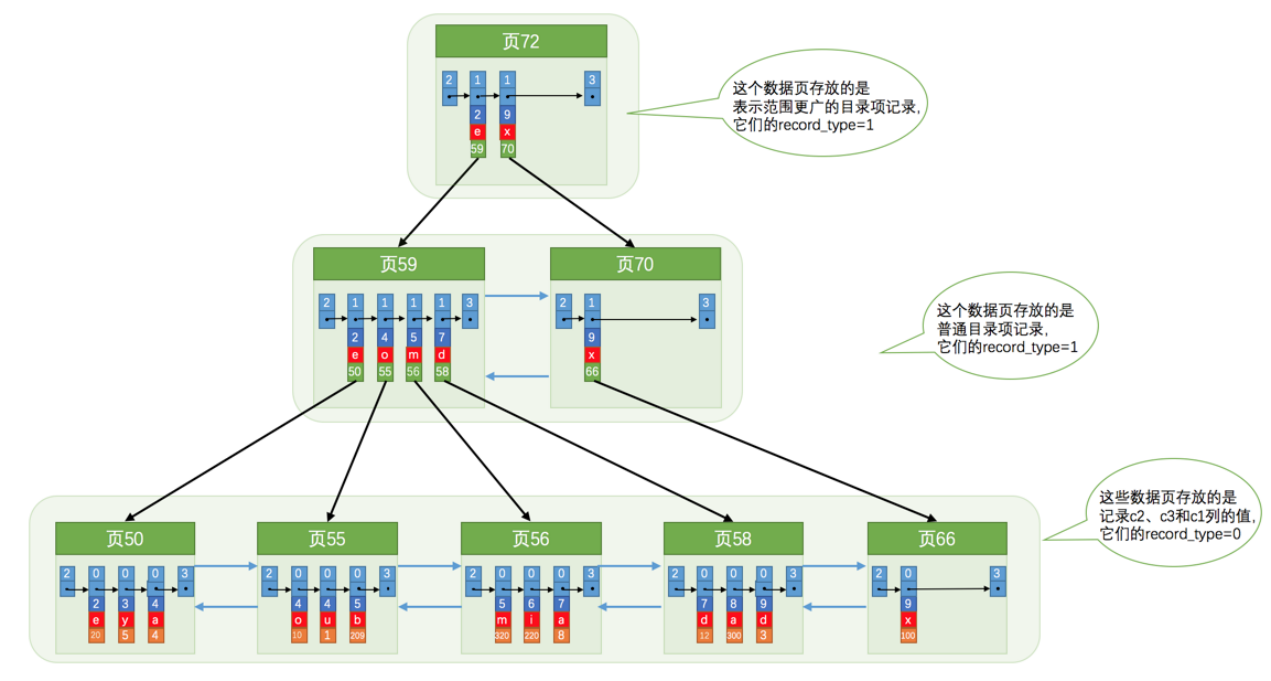
第 3 步就是回表。一次非主键查询需要遍历两棵 B+ 树。

为什么不在二级索引叶子节点直接存完整记录？因为每建一个索引就要复制一份全部数据，太浪费空间。

联合索引

同时以多个列作为排序规则建立的 B+ 树。例如为 `(c2, c3)` 建立联合索引：

- 先按 `c2` 排序。
- `c2` 相同时，再按 `c3` 排序。



联合索引 (c2, c3)

联合索引的结构：

- 目录项记录：`c2 + c3 +`
- 叶子节点记录：`c2 + c3 + c1`

最左前缀原则： 联合索引 `(c2, c3)` 能加速以下查询：

- `WHERE c2 = x`（用到 c2 排序）
- `WHERE c2 = x AND c3 = y`（同时用到 c2、c3 排序）

但不能单独加速 `WHERE c3 = y`，因为在 c2 不确定时 c3 并不是全局有序的。

联合索引 vs 多个单列索引：

- `INDEX(c2, c3)`：建 1 棵 B+ 树。
- `INDEX(c2)` + `INDEX(c3)`：建 2 棵 B+ 树。

两者的索引结构和适用查询完全不同。

InnoDB 中 B+ 树索引的注意事项

根页面固定不动

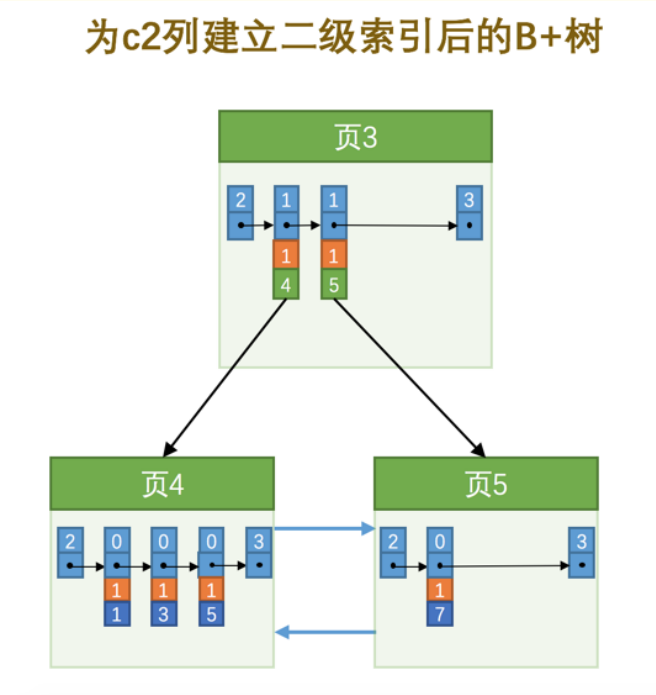
B+ 树的形成过程：

1. 创建索引时分配一个根节点页面，初始为空。
2. 插入记录时先存入根节点。
3. 根节点满后，将所有记录复制到新页 a，再分裂出新页 b，根节点升级为存储目录项记录的内节点。

核心规则：根节点页面自创建后不再移动。InnoDB
将根节点页号记录在数据字典中，通过固定入口访问整棵 B+ 树。

内节点目录项的唯一性

二级索引的内节点如果只存 `+` ，当索引列有大量重复值时，无法判断新记录该插入哪个子页。

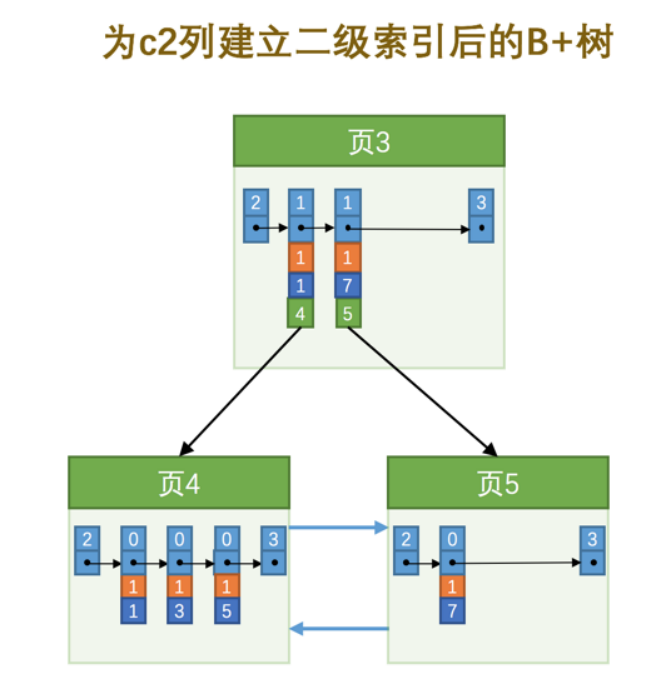


只有索引列+页号时的歧义

因此，二级索引内节点的目录项记录实际包含三部分：

- 索引列的值
- 主键值
- 页号

加入主键后保证同层内目录项记录的唯一性，消除定位歧义。



加入主键后的二级索引内节点

每个页面至少存 2 条记录

B+ 树高效的前提是每层能过滤大量子目录。如果每页只存 1 条记录，树的层级会急剧增长，退化为链表。因此 InnoDB 要求每个数据页至少存放 2 条记录。

InnoDB vs MyISAM 索引对比

对比项	InnoDB	MyISAM
数据存储	聚簇索引叶子节点	独立数据文件，按插入顺序
主键索引叶子	完整用户记录	主键 + 行号
二级索引叶子	索引列 + 主键	索引列 + 行号
聚簇索引	有，且唯一	无，所有索引都是二级索引
回表方式	主键 → 聚簇索引	行号 → 数据文件

核心区别：InnoDB 索引即数据；MyISAM 索引与数据分离。

创建和删除索引

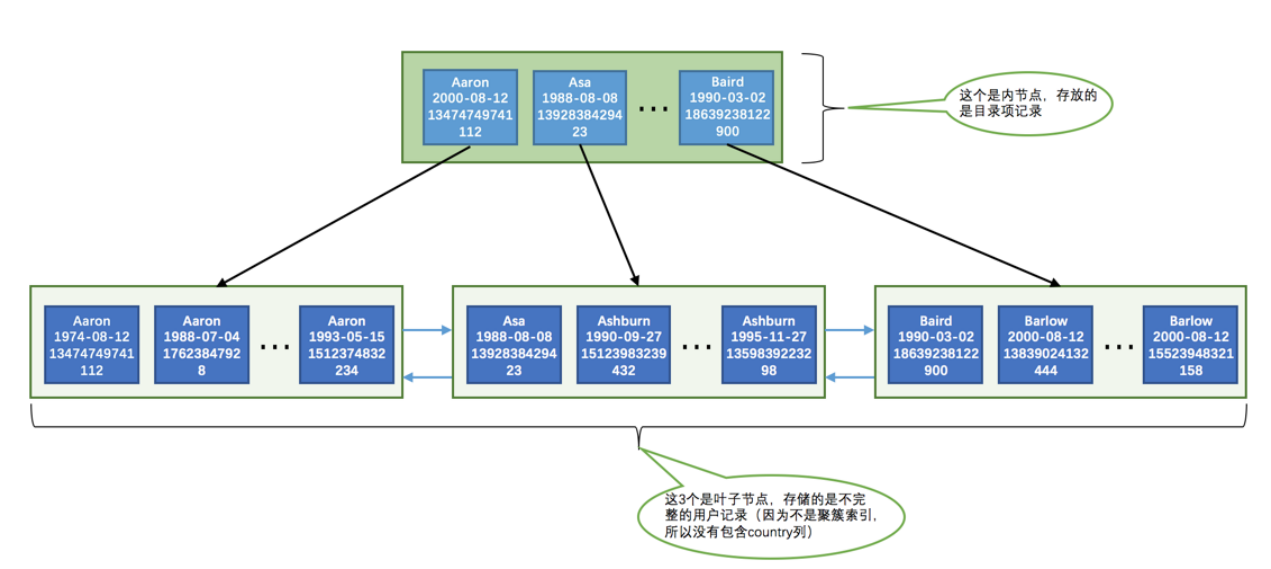
```sql
-- 建表时创建联合索引
CREATE TABLE t (
    id INT PRIMARY KEY,
    a INT, b CHAR(1),
    INDEX idx_a_b (a, b)
);

-- 添加/删除索引
ALTER TABLE t ADD INDEX idx_a (a);
ALTER TABLE t DROP INDEX idx_a;
```

InnoDB 自动为主键和 `UNIQUE` 列创建索引，其他列需手动创建。每多一个索引就多一棵 B+ 树，写入时需维护所有索引排序，不宜盲目建索引。

B+ 树索引的使用

本章基于联合索引 `idx_name_birthday_phone_number(name, birthday, phone_number)` 讨论索引使用规则。B+ 树按 `name` → `birthday` → `phone_number` 逐级排序。



联合索引 B+ 树示意

索引的代价

- 空间：每个索引对应一棵 B+ 树，索引越多占用空间越大。
- 时间：增删改需同步维护所有 B+ 树的排序，涉及记录移位、页分裂、页回收。索引越多写入越慢。

索引的使用条件

全值匹配

搜索条件包含联合索引的所有列：

```
SELECT * FROM person_info
WHERE name = 'Ashburn' AND birthday = '1990-09-27' AND phone_number = '15123983239';
```

三个列全部命中索引。WHERE 子句中列的书写顺序不影响索引使用——查询优化器会自动调整匹配顺序。

最左前缀匹配

只使用联合索引最左边的连续列，也能命中索引：

```
-- 命中 name
SELECT * FROM person_info WHERE name = 'Ashburn';
-- 命中 name + birthday
SELECT * FROM person_info WHERE name = 'Ashburn' AND birthday = '1990-09-27';
```

跳过最左列则无法使用索引：

```
-- 无法使用 idx_name_birthday_phone_number
SELECT * FROM person_info WHERE birthday = '1990-09-27';
```

原因：B+ 树先按 name 排序，name 不同时 birthday 无序。

跳过中间列时，只能用到中断前的列：

```
-- 只用到 name，phone_number 无法命中
SELECT * FROM person_info WHERE name = 'Ashburn' AND phone_number = '15123983239';
```

列前缀匹配

字符串按字符逐位排序，因此前缀是有序的：

```
-- 命中索引
SELECT * FROM person_info WHERE name LIKE 'As%';
-- 无法命中索引（前缀不确定）
SELECT * FROM person_info WHERE name LIKE '%As%';
```

范围查询与索引

索引记录本身有序，天然支持范围查询：

```
SELECT * FROM person_info WHERE name > 'Asa' AND name < 'Barlow';
```

但对联合索引的多列同时做范围查询时，只有最左列的范围条件能用到索引：

```
-- 只有 name 的范围条件用到索引，birthday 的范围条件无法使用
SELECT * FROM person_info WHERE name > 'Asa' AND name < 'Barlow' AND birthday > '1980-01-01';
```

原因：name 范围查找结果中 name 值不同，birthday 不保证有序。

精确匹配某列并范围匹配另一列

左边列精确匹配后，结果集在该列上值相同，右边列仍有序，可继续用索引做范围查找：

```
-- name 精确匹配 → birthday 范围命中 → phone_number 无法命中
SELECT * FROM person_info
WHERE name = 'Ashburn'
  AND birthday > '1980-01-01' AND birthday < '2000-12-31'
  AND phone_number > '15100000000';
```

name 精确匹配后结果按 birthday 排序，birthday 范围有效；但范围查找后 birthday 值不同，phone_number 无序，无法命中。若前两列都精确匹配，第三列也可范围查找：

```
-- 三列均命中索引
SELECT * FROM person_info
WHERE name = 'Ashburn' AND birthday = '1980-01-01' AND phone_number > '15100000000';
```

索引失效的场景

场景	示例	原因
索引列参与计算/函数	`WHERE my_col * 2 < 4`	引擎需逐行计算表达式，无法利用 B+ 树
跳过最左列	`WHERE birthday = '...'`	联合索引先按 name 排序，跳过则无序
跳过中间列	`WHERE name = 'A' AND phone = '...'`	只能用到 name
LIKE 左模糊	`WHERE name LIKE '%As%'`	前缀不确定，无法利用排序
排序列 ASC/DESC 混用	`ORDER BY name ASC, birthday DESC`	B+ 树单方向有序，混用无法直接读取
排序列不属于同一索引	`ORDER BY name, country`	不同索引无法协同排序
排序列使用函数	`ORDER BY UPPER(name)`	函数改变排序依据

核心原则：索引列必须以单独列的形式出现在比较表达式中。`WHERE my_col < 4/2` 可以用索引，`WHERE my_col * 2 < 4` 不行。

回表的代价与覆盖索引

回表过程

1. 在二级索引 B+ 树中找到符合条件的记录（顺序 I/O，索引记录物理连续）。
2. 取出每条记录的主键 `id`，到聚簇索引中查完整记录（随机 I/O，`id` 值不连续，完整记录分散在不同页）。

回表记录越多，随机 I/O 开销越大。当回表行数占比过高时，优化器宁愿全表扫描也不走二级索引。添加 `LIMIT` 可减少回表量，促使优化器选择索引。

覆盖索引

查询列全部包含在索引中，无需回表：

```
-- 覆盖索引：只查 name, birthday, phone_number
SELECT name, birthday, phone_number FROM person_info
WHERE name > 'Asa' AND name < 'Barlow';
```

排序查询同理——覆盖索引可避免 filesort 和回表的双重开销：

```
SELECT name, birthday, phone_number FROM person_info
ORDER BY name, birthday, phone_number;
```

避免 `SELECT *`，只查需要的列，尽量覆盖索引。

索引用于排序

filesort vs 索引排序

- filesort：将记录加载到内存（或借助磁盘临时文件）进行排序，代价高。
- 索引排序：B+ 树本身有序，直接按序读取，无需额外排序。

```
-- 索引排序：ORDER BY 顺序与索引列顺序一致
SELECT * FROM person_info ORDER BY name, birthday, phone_number LIMIT 10;
```

左边列为常量时，后续列也可用于索引排序：

```
-- name 固定后，结果集按 birthday, phone_number 排序
SELECT * FROM person_info WHERE name = 'A' ORDER BY birthday, phone_number LIMIT 10;
```

不能使用索引排序的场景

1. 排序列顺序与索引列顺序不一致：`ORDER BY phone_number, birthday, name` 无法使用索引。
2. ASC/DESC 混用：`ORDER BY name ASC, birthday DESC` 无法使用索引。B+ 树全升序或全降序扫描均可，但混合方向不行。
3. WHERE 使用了非索引列过滤：`WHERE country = 'China' ORDER BY name` 无法使用索引排序（`country` 不在索引中）。
4. 排序列来自不同索引：`ORDER BY name, country` 中两列不属于同一索引。
5. 排序列使用表达式或函数：`ORDER BY UPPER(name)` 无法使用索引。

索引用于分组

GROUP BY 原理与排序类似。B+ 树索引已排好序，相同值自然聚集，可直接分组：

```
SELECT name, birthday, phone_number, COUNT(*)
FROM person_info
GROUP BY name, birthday, phone_number;
```

规则与排序相同：分组列顺序必须与索引列顺序一致，可只使用左边的连续列。

索引设计原则

只为搜索、排序、分组的列建索引

只为 `WHERE`、`JOIN`、`ORDER BY`、`GROUP BY` 中出现的列创建索引。查询列表中的列不需要单独建索引。

选择区分度高的列

列的基数（不重复值个数）越大，索引越有效。基数极低的列建索引无意义，且会产生大量回表。

索引列类型尽量小

类型越小，比较越快，单页容纳更多记录，减少 I/O。对主键尤为重要——主键值存储在所有二级索引中。能用 `INT` 就不用 `BIGINT`。

使用前缀索引

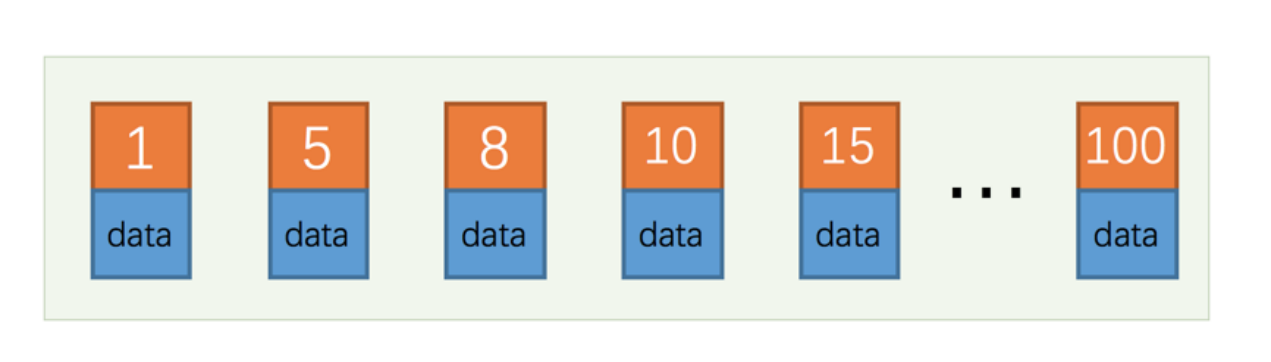
对长字符串只索引前 N 个字符，节省空间并加速比较：

```
KEY idx_name_birthday_phone_number (name(10), birthday, phone_number)
```

代价：前缀索引无法支持索引排序。

主键自增

主键值递增插入时，记录依次追加到页尾，不会触发页分裂。主键值若忽大忽小，插入已满的页面会导致页分裂和记录移位，带来性能损耗。



页分裂示意

建议主键使用 `AUTO_INCREMENT`：

```
id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY
```

避免冗余和重复索引

联合索引 `(name, birthday, phone_number)` 已覆盖对 `name` 的搜索，无需再单独建索引。同一列同时定义主键、唯一索引、普通索引属于重复索引，应清理。

优先覆盖索引

设计索引时考虑常用查询列，让查询只访问索引即可获取全部所需数据。

Buffer Pool

什么是 Buffer Pool

InnoDB 的数据以页（16KB）为单位存储在磁盘上。磁盘 IO 远慢于 CPU，因此 InnoDB 在启动时向操作系统申请一块连续的内存空间作为缓存，称为 Buffer Pool。访问某页数据时，先将整页加载到 Buffer Pool，后续对该页的访问直接走内存，省去磁盘 IO。

即使只访问页中的一条记录，也需要将整个页加载到内存。读写完成后不立即释放，而是缓存起来供后续请求复用。

默认大小 128MB，通过 `innodb_buffer_pool_size` 配置，最小值 5MB：

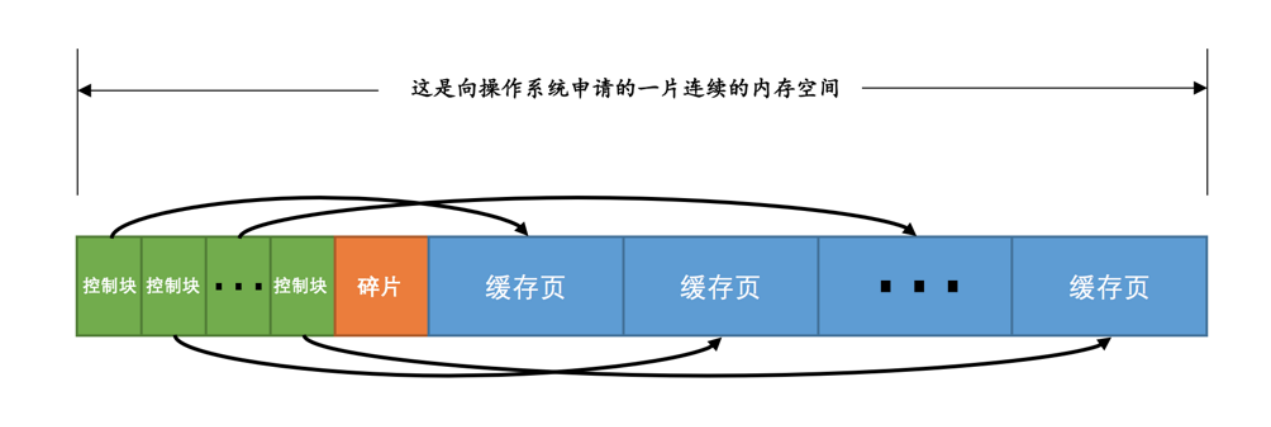
```
[server]
innodb_buffer_pool_size = 268435456  -- 256MB
```

Buffer Pool 内部结构

控制块与缓存页

Buffer Pool 中缓存页大小与磁盘页一致（16KB）。每个缓存页都有一个对应的控制块，记录表空间编号、页号、缓存页地址、链表指针、锁信息、LSN 等元信息。控制块约占缓存页大小的 5%（MySQL 5.7.21 中为 808 字节）。

内存布局：控制块在前，缓存页在后，中间可能产生不足一对的碎片。

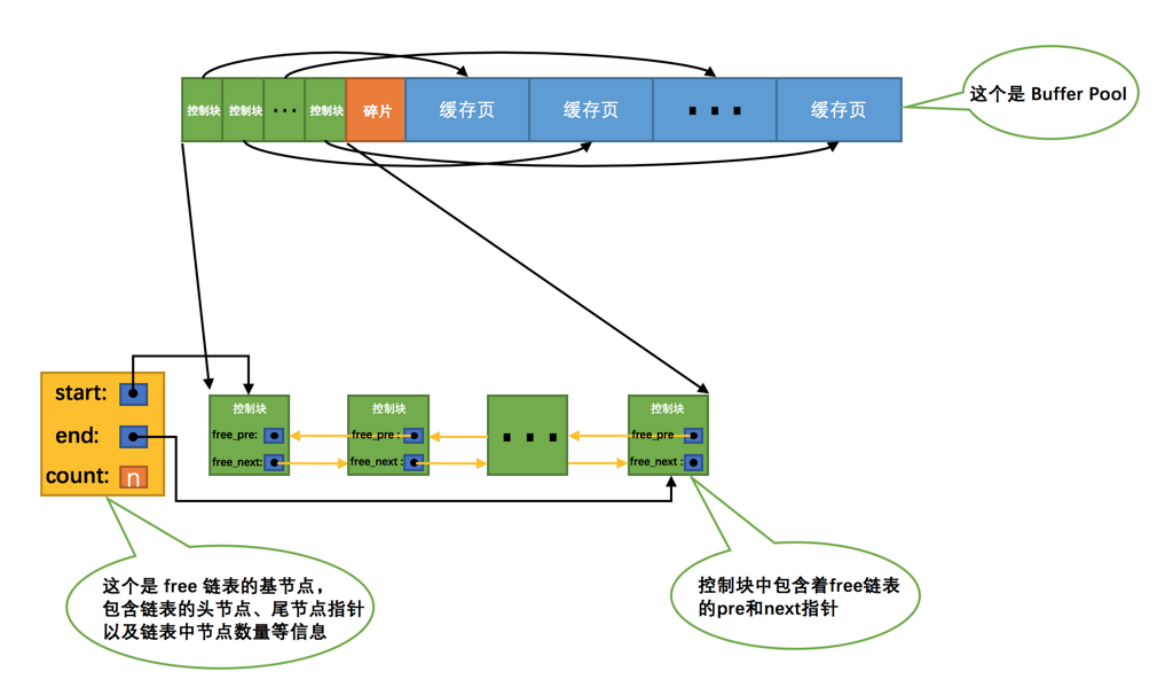


> `innodb_buffer_pool_size` 不包含控制块的空间，InnoDB 实际申请的内存约比配置值大 5%。

free 链表

启动时所有缓存页都空闲，其控制块组成一个双向链表——free 链表。链表有一个独立分配的基节点（40 字节），记录头尾指针和节点数。

从磁盘加载页时：从 free 链表取一空闲缓存页 → 填充控制块信息 → 从 free 链表中移除该节点。



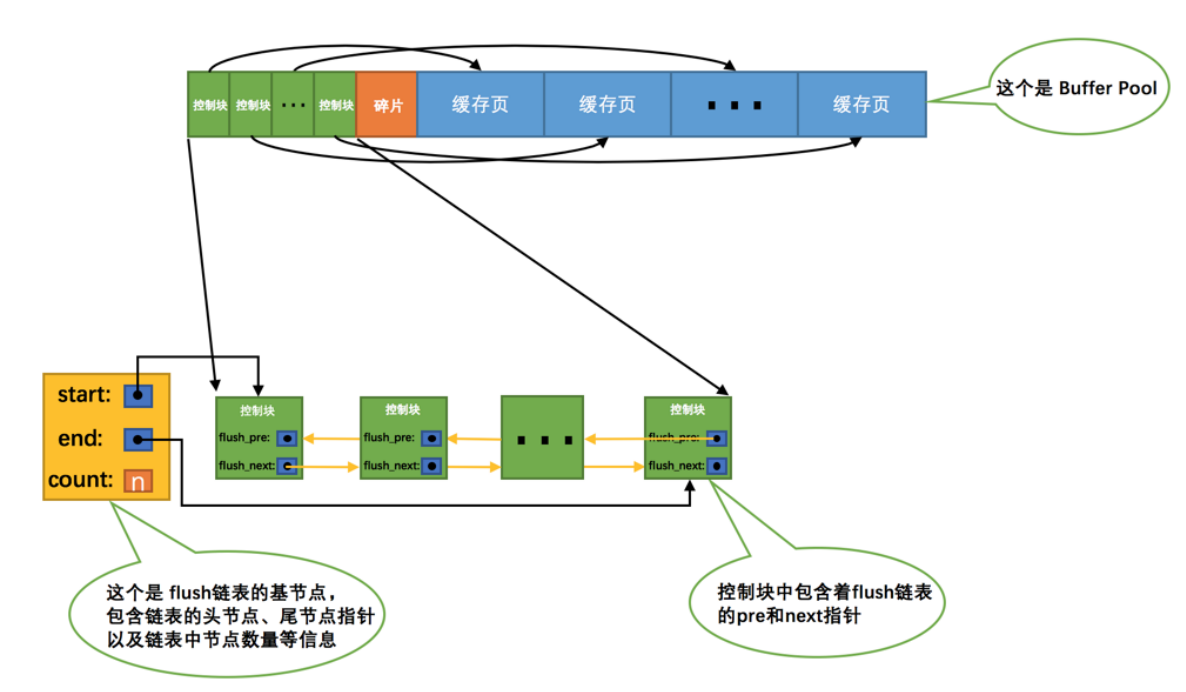
缓存页的哈希表

如何判断某页是否已在 Buffer Pool 中？遍历太慢。InnoDB 用 `+` 作 key，缓存页作 value 建哈希表，O(1) 定位。访问某页时先查哈希表：命中则直接使用；未命中则从 free 链表分配缓存页，将磁盘数据加载进来，同时在哈希表中插入映射。

`flush` 链表

修改了 Buffer Pool 中某缓存页的数据后，该页与磁盘不一致，称为脏页（dirty page）。脏页不会立即刷盘，而是将其控制块加入 `flush` 链表，等后台线程择机同步。

flush 链表结构与 free 链表类似，只是节点是所有被修改过的缓存页。



LRU 链表

Buffer Pool 容量有限，free 链表耗尽时需要淘汰旧页。InnoDB 用 LRU（Least Recently Used）链表管理淘汰策略。

简单 LRU 的问题

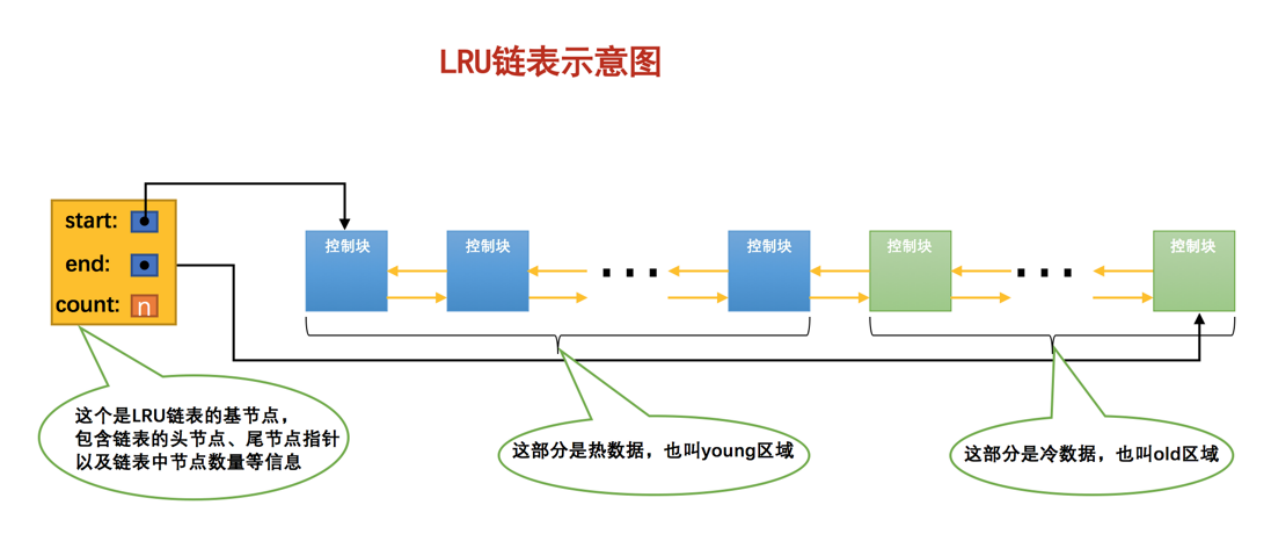
朴素做法：访问一页时将其移到链表头部，淘汰时从尾部移除。但有两个场景会严重拉低缓存命中率：

1. 预读（read ahead）：InnoDB 可能预先加载后续页面到 Buffer Pool。如果预读页未被实际访问，却占据了链表头部，就会把真正的热页挤到尾部被淘汰。 - 线性预读：顺序访问某个区（extent）的页面超过 `innodb_read_ahead_threshold`（默认 56）时，异步预读下一个区的全部页面。 - 随机预读：Buffer Pool 中已缓存某区 13 个连续页面时触发（默认关闭，`innodb_random_read_ahead = OFF`）。
2. 全表扫描：一次性加载大量页面，把整个 Buffer Pool 换血，其他热页全被淘汰。而全表扫描本身执行频率很低。

分区 LRU（young 区 + old 区）

InnoDB 将 LRU 链表分为两段：

区域	含义	默认占比
young 区	热数据，使用频率高的缓存页	约 63%
old 区	冷数据，使用频率低的缓存页	约 37%



`innodb_old_blocks_pct` 控制 old 区比例，默认 37（即 37%）。可动态修改：

```
SET GLOBAL innodb_old_blocks_pct = 40;
```

分区后的核心规则：

- 新页首次加载时放入 old 区头部，而非整条链表头部。预读但未使用的页自然从 old 区淘汰，不影响 young 区。
- old 区的页被再次访问时，若距首次访问的时间间隔超过 `innodb_old_blocks_time`（默认 1000ms），才移动到 young 区头部。全表扫描中同一页的多次访问通常在 1s 内完成，因此不会进入 young 区。

预读与全表扫描的优化

分区 LRU + `innodb_old_blocks_time` 两道防线：

1. 预读页只进 old 区，用不到自然被淘汰。
2. 全表扫描的页虽在短时间内被多次访问，但因时间间隔不足而留在 old 区。

进一步优化：young 区内的缓存页，如果位于 young 区前 1/4，再次访问时不移动到头部，降低链表操作频率。

当 Buffer Pool 中无空闲缓存页时，从 LRU 链表尾部（old 区末端）淘汰最冷的页面。

刷脏页

后台线程负责将脏页异步刷盘，主要有三种方式：

方式	来源	说明
BUF_FLUSH_LRU	LRU 链表尾部	定时从尾部扫描 `innodb_lru_scan_depth` 个页面，发现脏页就刷盘
BUF_FLUSH_LIST	flush 链表	定时从 flush 链表刷一批脏页，速率随系统负载调节

BUF_FLUSH_SINGLE_PAGE	LRU 链表尾部	用户线程找不到空闲页时，同步刷 LRU 尾部的一个脏页（影响性能）

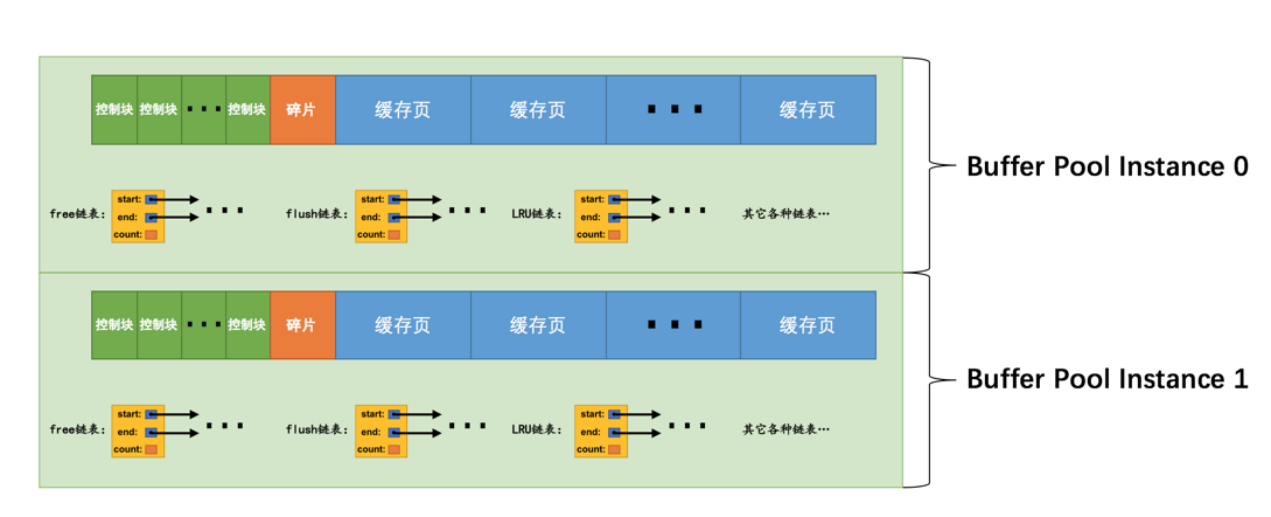
第三种是兜底策略——后台线程刷新不够快，用户线程被迫同步刷盘，会显著拖慢请求处理速度。

系统特别繁忙时，用户线程甚至会批量从 flush 链表刷脏页，这与 redo log 的 checkpoint 机制有关。

多 Buffer Pool 实例

多线程并发访问 Buffer Pool 需要加锁。当 Buffer Pool 很大时，单实例锁竞争成为瓶颈。InnoDB 支持拆分为多个独立实例，各自维护 free、flush、LRU 链表，互不干扰。

```
[server]
innodb_buffer_pool_instances = 2
```

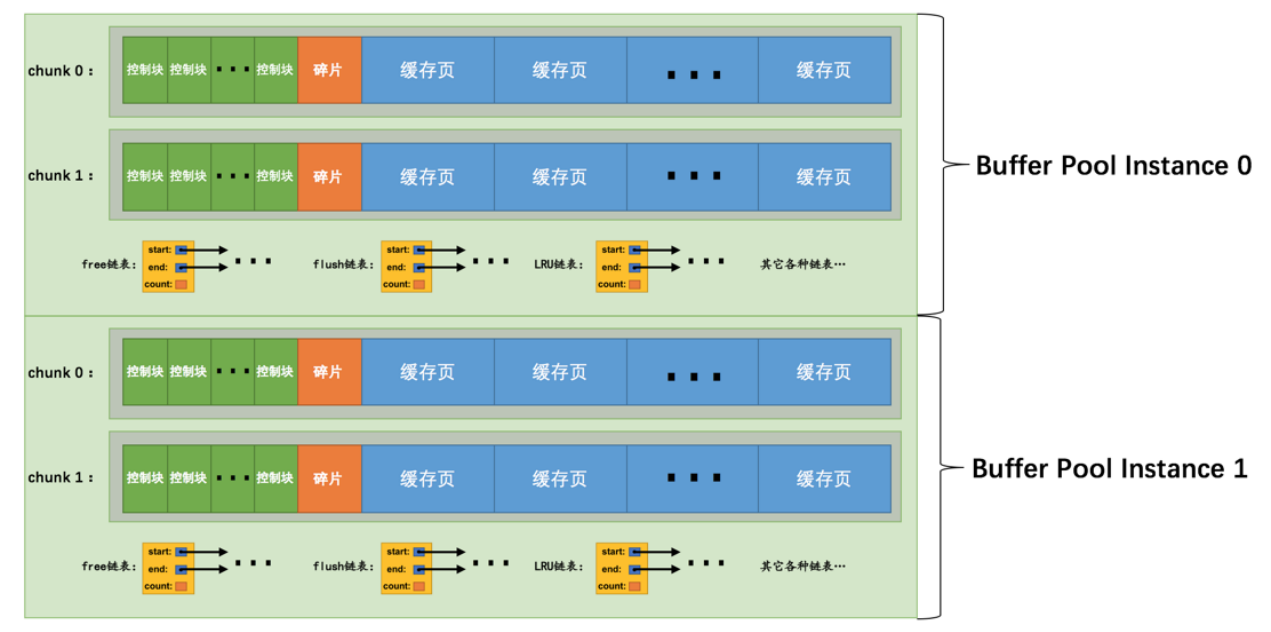


每个实例大小 = `innodb_buffer_pool_size / innodb_buffer_pool_instances`。

> 当 `innodb_buffer_pool_size < 1GB` 时，多实例设置无效，InnoDB 强制使用 1 个实例。

chunk 机制

MySQL 5.7.5 起支持运行时动态调整 Buffer Pool 大小。为避免重新申请整块连续内存，引入 chunk 概念——每个实例由若干 chunk 组成，每个 chunk 是一片连续内存（默认 128MB，`innodb_buffer_pool_chunk_size` 启动时指定，运行时不可更改）。



配置约束：

- `innodb_buffer_pool_size` 必须是 `chunk_size × instances` 的整数倍，否则自动向上取整。
- 若 `chunk_size × instances > buffer_pool_size`，chunk_size 会被自动调整为 `buffer_pool_size / instances`。

查看 Buffer Pool 状态

```
SHOW ENGINE INNODB STATUS\G
```

关键指标：

字段	含义
Buffer pool size	可容纳的缓存页数（单位：页）
Free buffers	free 链表剩余节点数
Database pages	LRU 链表总页数
Old database pages	LRU old 区页数
Modified db pages	flush 链表节点数（脏页数）
Buffer pool hit rate	缓存命中率（X / 1000）
Pages made young	从 old 区移入 young 区的累计次数
youngs/s / non-youngs/s	每秒移入 young 区 / 因时间限制未移入的次数

其他信息

Buffer Pool 除了缓存数据页，还存储锁信息和自适应哈希索引（Adaptive Hash Index）。自适应哈希索引由 InnoDB 自动为热点页建立，将 B+ 树的查找从 O(log n) 优化为 O(1)，无需手动干预。

第二部分：查询优化

单表访问方法

MySQL 查询优化器在执行单表查询时，会选择不同的访问方法（access method）来获取数据。访问方法的选择直接决定查询性能。

下文以 `single_table` 表为例。索引：聚簇索引（id）、普通二级索引 idx_key1（key1）、唯一二级索引 idx_key2（key2）、普通二级索引 idx_key3（key3）、联合索引 idx_key_part（key_part1, key_part2, key_part3）。

访问方法概述

单表查询的执行方式分两大类：

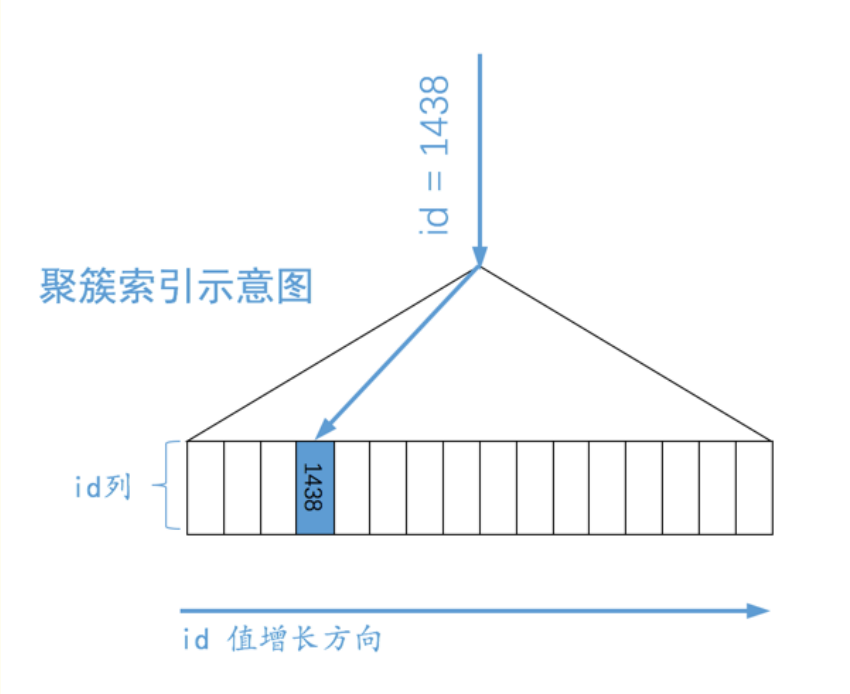
- 全表扫描：逐行遍历聚簇索引，逐条匹配搜索条件。代价最高。
- 索引查询：利用索引快速定位记录，细分为以下几种访问方法。

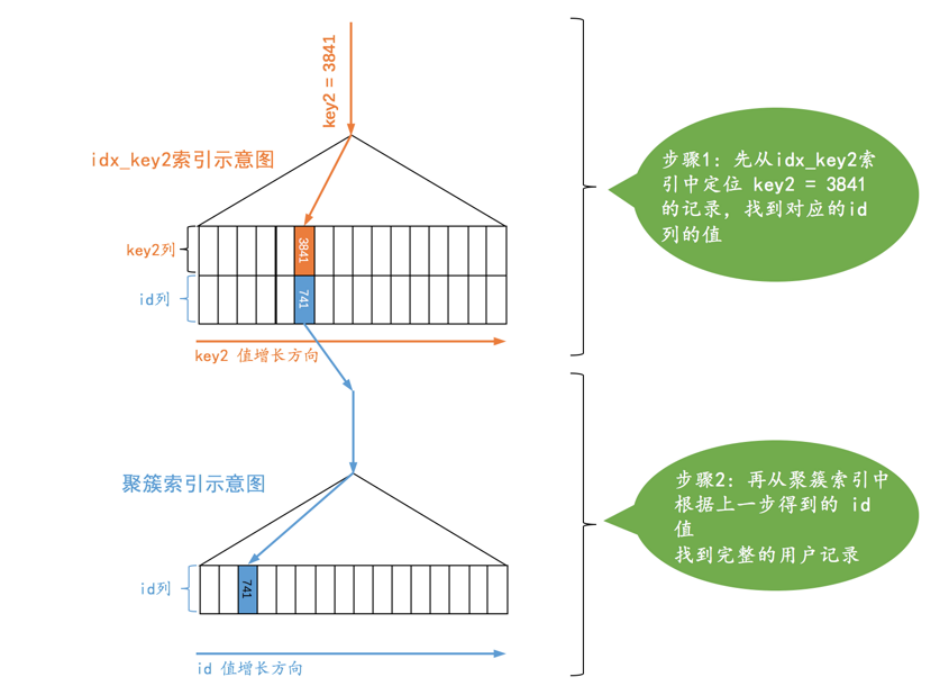
性能排序（由优到劣）：`const` > `ref` > `ref_or_null` > `range` > `index` > `ALL`

const

通过主键或唯一二级索引与常数进行等值比较，最多定位一条记录。

```
SELECT * FROM single_table WHERE id = 1438;
SELECT * FROM single_table WHERE key2 = 3841;
```



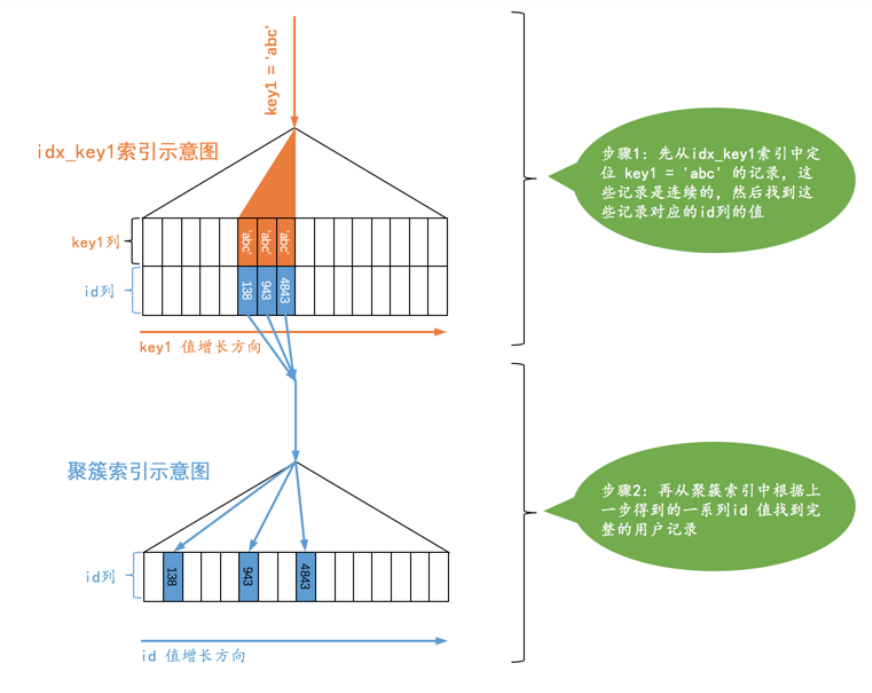


- 主键直接在聚簇索引定位；唯一二级索引先定位再回表。
- 联合唯一索引必须所有列都等值匹配才能用 const。
- `WHERE key2 IS NULL` 不能用 const——唯一索引不限制 NULL 数量，可能匹配多行。

ref

通过普通二级索引与常数进行等值比较。可能匹配多条连续记录，需逐一回表。

```
SELECT * FROM single_table WHERE key1 = 'abc';
```

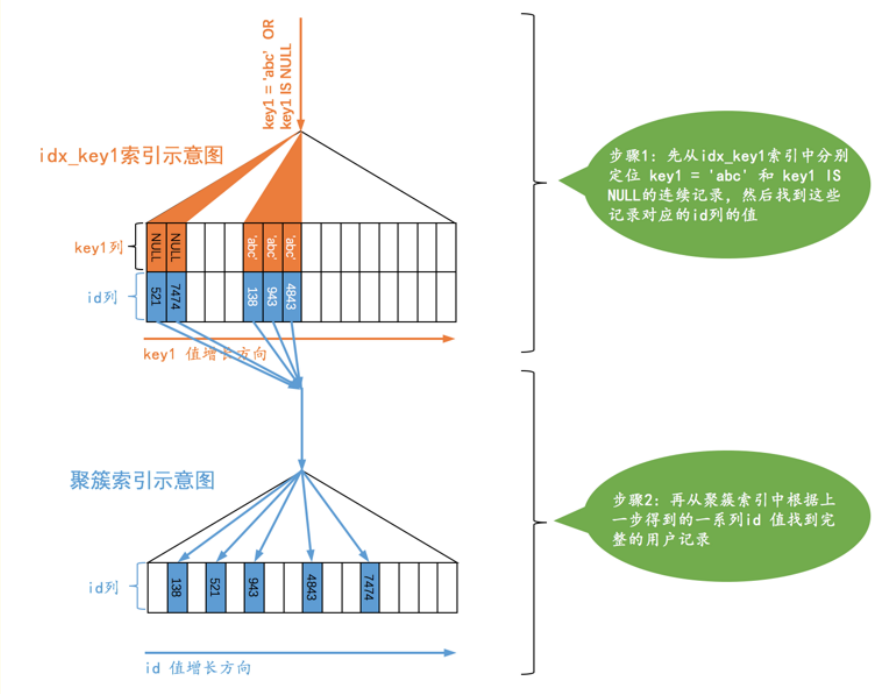


- 效率取决于匹配的记录条数，越少回表代价越低。
- `key IS NULL` 最多用 ref，不能用 const。
- 联合索引只要最左连续列都是等值匹配即可用 ref。若中间出现范围匹配则不行。

ref_or_null

在 ref 基础上额外检索索引列为 NULL 的记录。

```
SELECT * FROM single_table WHERE key1 = 'abc' OR key1 IS NULL;
```

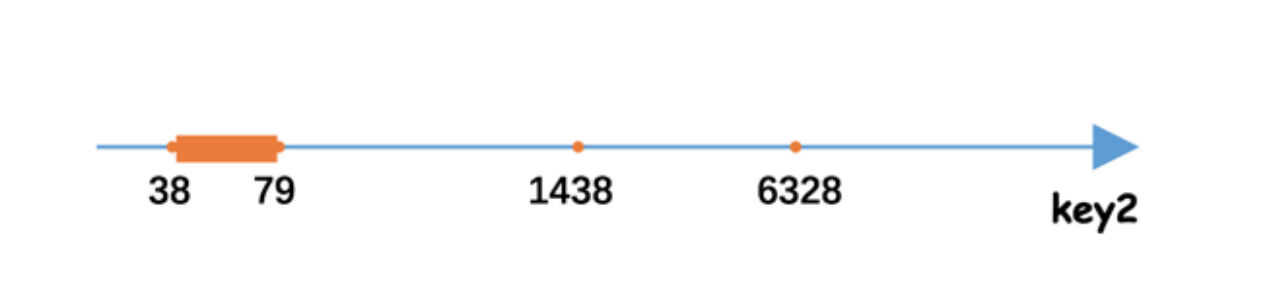


相当于在二级索引 B+ 树中找出 `key1 IS NULL` 和 `key1 = 'abc'` 两个连续范围的记录，再回表。

range

索引列与常数进行范围匹配，包括 `=`、`IN`、`IS NULL`、`>`、`<`、`>=`、`<=`、`BETWEEN`、`!=`、`LIKE` 等操作符产生的区间。

```
SELECT * FROM single_table WHERE key2 IN (1438, 6328) OR (key2 >= 38 AND key2 <= 79);
```



上述查询产生三个范围区间：`key2 = 1438`（单点区间）、`key2 = 6328`（单点区间）、`key2 ∈ [38, 79]`（连续范围区间）。

范围区间的确定规则：

- 多条件用 AND 连接 → 取各区间交集
- 多条件用 OR 连接 → 取各区间并集
- 无法使用当前索引的搜索条件替换为 TRUE 后化简
- 若化简结果为 `(-∞, +∞)` 则该索引不值得使用

典型场景：一个索引条件 OR 一个非索引条件 → 化简后区间为全部，不走索引。

```
-- 化简: key2 > 100 OR TRUE → TRUE → 全表扫描
SELECT * FROM single_table WHERE key2 > 100 OR common_field = 'abc';
```

index

遍历二级索引的全部叶子节点，不回表。适用条件：

1. 查询列全部包含在某个二级索引中（覆盖索引）
2. 搜索条件列也包含在该索引中

```
SELECT key_part1, key_part2, key_part3 FROM single_table WHERE key_part2 = 'abc';
```

`key_part2` 不是联合索引最左列，无法用 ref/range，但查询列和条件列都在 idx_key_part 中。直接遍历二级索引比遍历聚簇索引代价小得多（二级索引记录更紧凑，且无需回表）。

ALL

全表扫描，直接遍历聚簇索引所有记录。所有查询都可以用这种方式执行，但代价最高。

二级索引 + 回表的执行流程

一般情况下一条查询只使用一个二级索引。以 `WHERE key1 = 'abc' AND key2 > 1000` 为例：优化器选扫描行数更少的索引（假设 idx_key1）→ 取出匹配记录 → 回表取完整行 → 用剩余条件 `key2 > 1000` 过滤。

关键点：索引定位阶段只能用当前索引相关的条件，其余条件在回表后才能过滤。

Index Merge（索引合并）

特殊情况下，MySQL 可以同时使用多个二级索引完成查询，称为索引合并。

Intersection（交集合并）

适用于多个搜索条件用 AND 连接的场景。

```
SELECT * FROM single_table WHERE key1 = 'a' AND key3 = 'b';
```

执行过程：分别从 idx_key1 和 idx_key3 取出主键值集合 → 求主键交集 → 按交集结果回表。

使用条件：

- 二级索引列必须等值匹配（联合索引每列都要匹配完整）
- 主键列可以范围匹配（直接在二级索引结果上过滤）

原理： 等值匹配时二级索引结果集天然按主键排序，两个有序集合求交集只需 O(n) 归并。非等值匹配则主键无序，排序代价太高。按有序主键回表称为 ROR（Rowid Ordered Retrieval）。

Union（并集合并）

适用于多个搜索条件用 OR 连接的场景。

```
SELECT * FROM single_table WHERE key1 = 'a' OR key3 = 'b';
```

使用条件与 Intersection 类似：二级索引列等值匹配、主键可范围匹配。另外搜索条件的部分子集可以先 Intersection 合并再参与 Union。原理相同——等值匹配保证主键有序，求并集 O(n)。

Sort-Union（排序并集合并）

放宽了 Union 的等值匹配限制，允许范围条件。

```
SELECT * FROM single_table WHERE key1 < 'a' OR key3 > 'z';
```

执行过程：分别从各二级索引取出记录 → 按主键排序 → 按 Union 方式求并集 → 回表。

多了一步主键排序，但因为范围条件获取的记录通常不多，排序代价可接受。

> 为什么没有 `Sort-Intersection`？ Intersection 的场景是单索引匹配记录太多才需要合并减少回表量。而大量记录的排序代价可能比直接回表还高，因此没有引入 Sort-Intersection。

注意事项

- 索引合并发生的条件是必要条件而非充分条件，最终由优化器根据成本估算决定。
- 如果查询经常触发 Intersection 合并，优先考虑建联合索引替代多个单列索引，避免多棵 B+ 树的额外开销。
- 索引列 OR 非索引列的条件组合无法使用该索引（化简后区间为全集）。
- 回表操作是随机 `I/O`，二级索引扫描是顺序 `I/O`。当回表记录数过多时，全表扫描可能反而更快。

连接的原理

连接的本质

连接查询就是把各个表中的记录取出来依次匹配组合，加入结果集返回。两张表直接连接产生的结果集称为笛卡尔积——t1 有 3 条记录、t2 有 3 条记录，笛卡尔积就是 3×3=9 行。

```
SELECT * FROM t1, t2;  -- 产生笛卡尔积
```

实际使用中必须加过滤条件来减少结果集，过滤条件分两类：

- 单表条件：只涉及一张表，如 `t1.m1 > 1`
- 两表条件：涉及两张表，如 `t1.m1 = t2.m2`

连接查询的执行步骤：

1. 确定驱动表，用单表条件对驱动表执行查询，得到结果集
2. 对结果集中的每一条记录，带入两表条件去被驱动表中查找匹配记录

驱动表只访问一次，被驱动表的访问次数取决于驱动表结果集的记录条数。

连接类型

内连接

驱动表中的记录在被驱动表中找不到匹配记录时，不会加入结果集。

MySQL 中以下写法等价：

```
SELECT * FROM t1, t2 WHERE t1.m1 = t2.m2;
SELECT * FROM t1 JOIN t2 ON t1.m1 = t2.m2;
SELECT * FROM t1 INNER JOIN t2 ON t1.m1 = t2.m2;
SELECT * FROM t1 CROSS JOIN t2 ON t1.m1 = t2.m2;
```

内连接中 `ON` 子句和 `WHERE` 子句等价，驱动表和被驱动表可以互换，不影响结果。

左/右外连接

驱动表中的记录即使在被驱动表中没有匹配记录，仍然加入结果集，被驱动表对应字段用 `NULL` 填充。

```
-- 左外连接：左表为驱动表
SELECT * FROM t1 LEFT JOIN t2 ON t1.m1 = t2.m2;

-- 右外连接：右表为驱动表
SELECT * FROM t1 RIGHT JOIN t2 ON t1.m1 = t2.m2;
```

以 t1 = {1,2,3}、t2 = {2,3,4} 为例，三种连接结果对比：

连接类型	结果行数	特殊行
INNER JOIN	2	无

LEFT JOIN	3	t1 中 m1=1 的记录保留，t2 列为 NULL
RIGHT JOIN	3	t2 中 m2=4 的记录保留，t1 列为 NULL

外连接的驱动表和被驱动表不能互换，左连接驱动表固定为左表，右连接固定为右表。外连接必须使用 `ON` 子句指定连接条件。

ON 子句与 WHERE 子句的区别

两者在外连接中语义不同：

- WHERE 子句：不论内连接还是外连接，不符合条件的记录一律从结果集中剔除
- ON 子句：对于外连接，驱动表记录在被驱动表中找不到满足 ON 条件的匹配时，该记录仍保留在结果集中，被驱动表列用 NULL 填充

对于内连接，ON 和 WHERE 完全等价。

实践建议：单表过滤条件放 WHERE，两表连接条件放 ON。

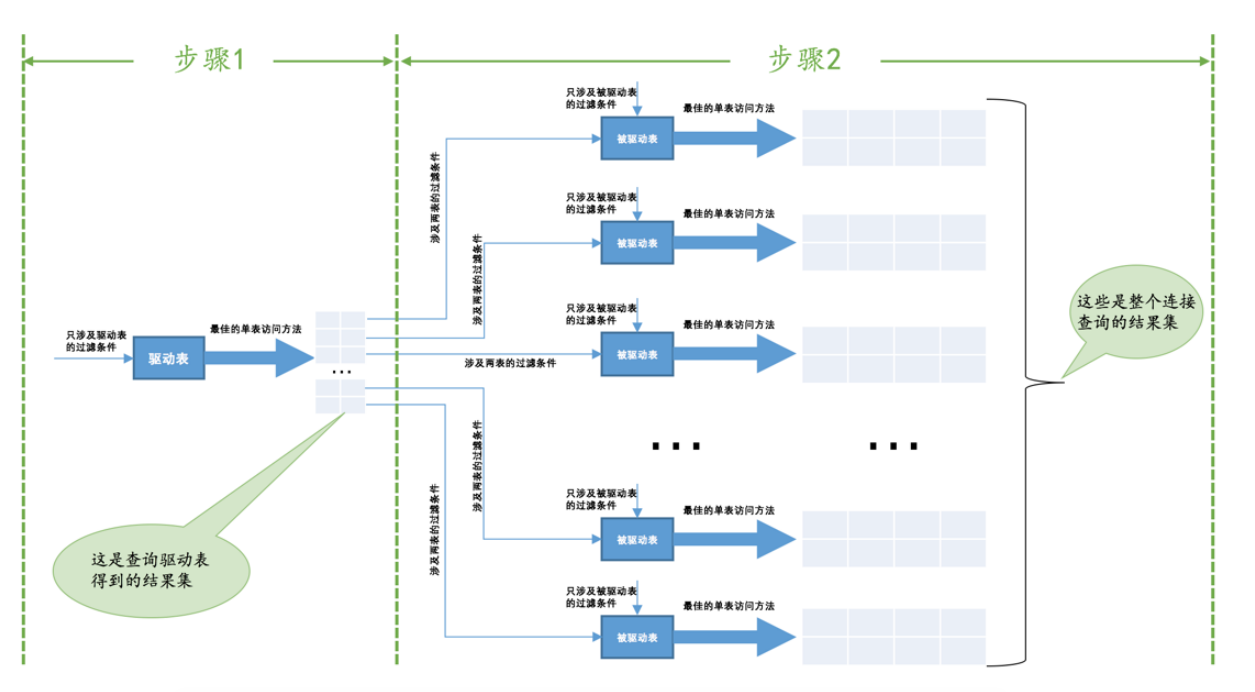
连接的执行原理

嵌套循环连接（Nested-Loop Join）

最基本的连接算法。伪代码：

```
for each row in t1 {            -- 遍历驱动表结果集
    for each row in t2 {        -- 遍历被驱动表
        if row satisfies join conditions, send to client
    }
}
```

三表连接则再嵌套一层循环。驱动表只访问一次，被驱动表访问次数 = 驱动表结果集行数。



嵌套循环连接示意图

这种算法简单但效率低——如果被驱动表无索引，每次都要全表扫描。

使用索引加速连接

被驱动表的每次查询本质上就是一次单表查询，可以利用索引加速。例如：

```
SELECT * FROM t1, t2 WHERE t1.m1 > 1 AND t1.m1 = t2.m2 AND t2.n2 < 'd';
```

对 t2 的查询相当于 `SELECT * FROM t2 WHERE t2.m2 = 2 AND t2.n2 < 'd'`，可利用的索引策略：

索引列	访问方法	说明
m2	ref	等值查找，回表后判断 n2 条件
m2（主键/唯一索引）	eq_ref	等值查找代价为常数级
n2	range	范围查找，回表后判断 m2 条件

`eq_ref` 是连接查询特有的访问方法：对被驱动表使用主键或唯一二级索引列做等值查找。

两个索引都可用时，优化器选择代价更低的那个。索引代价高于全表扫描时不会使用索引。

如果查询列表和过滤条件涉及的列恰好都属于某个索引，还可以走索引覆盖扫描（index 访问方法），避免回表。因此不要用 `SELECT *`，只查需要的列。

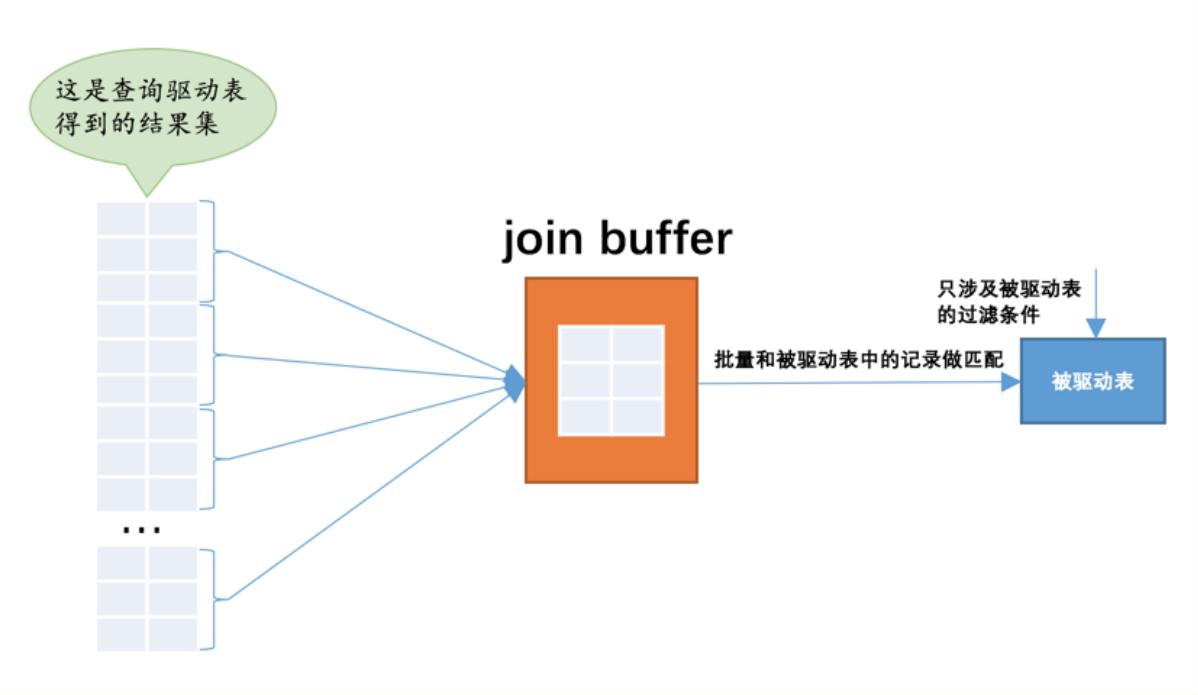
基于块的嵌套循环连接（Block Nested-Loop Join）

普通嵌套循环连接中，驱动表结果集有多少条记录，被驱动表就要从磁盘加载多少次，I/O 代价极高。

优化思路：把多条驱动表记录一起加载到内存，一次性和被驱动表记录做批量匹配，减少被驱动表的磁盘读取次数。

MySQL 引入了 `join buffer`：

1. 将若干条驱动表记录装入 join buffer
2. 扫描被驱动表，每条记录和 join buffer 中的所有驱动表记录做匹配
3. 匹配完成后清空 buffer，装入下一批驱动表记录，重复步骤 2



Block Nested-Loop Join 示意图

理想情况下 join buffer 能容纳驱动表全部结果集，被驱动表只需扫描一次。

`join_buffer_size`

- 通过系统变量 `join_buffer_size` 配置，默认 256KB（262144 字节）
- 最小值 128 字节
- join buffer 中只存放查询列表中的列和过滤条件中的列，不是整行数据

因此 `SELECT *` 不仅浪费网络带宽，还会让 join buffer
中能存放的记录数减少，间接增加被驱动表的扫描次数。

优化优先级：为被驱动表建立合适的索引 > 调大 join_buffer_size。

驱动表的选择

- 内连接：优化器自由选择驱动表，通常选结果集较小的表作为驱动表（减少被驱动表的访问次数）
- 左外连接：驱动表固定为左表
- 右外连接：驱动表固定为右表

核心原则：小表驱动大表。这里的"小表"指的是经过过滤条件筛选后结果集行数更少的表，而非原始数据量更少的表。

查询优化规则

MySQL 查询优化器会在执行前对语句进行查询重写，通过一系列规则将低效语句转换为高效形式。

条件化简

优化器会自动简化 WHERE 条件表达式：

- 移除不必要的括号：去掉多余嵌套括号
- 常量传递：`a = 5 AND b > a` → `a = 5 AND b > 5`（仅 AND 连接时有效，OR 不行）
- 等值传递：`a = b AND b = c AND c = 5` → `a = 5 AND b = 5 AND c = 5`
- 移除恒真/恒假条件：`b = b` → TRUE，`5 != 5` → FALSE，然后继续简化
- 常量折叠：`a = 5 + 1` → `a = 6`（仅限纯常量表达式）
- HAVING 合并：无聚集函数和 GROUP BY 时，HAVING 合并到 WHERE
- 常量表检测：主键等值匹配或唯一索引等值匹配的表被视为常量表，优先执行后用常量替换相关条件

> 面试要点：索引列必须以单独形式出现才能使用索引。`ABS(a) > 5` 或 `-a < -8` 都无法使用索引。

外连接消除

外连接的驱动表和被驱动表位置固定，无法像内连接那样自由调整连接顺序来优化成本。

核心规则：空值拒绝（Reject-NULL）

当 WHERE 子句中包含被驱动表的列不为 NULL 的条件时，外连接等价于内连接：

```
-- 以下左连接等价于内连接（WHERE 隐含拒绝 NULL）
SELECT * FROM t1 LEFT JOIN t2 ON t1.m1 = t2.m2 WHERE t2.m2 = 2;
-- 等价于
SELECT * FROM t1 INNER JOIN t2 ON t1.m1 = t2.m2 WHERE t2.m2 = 2;
```

不需要显式写 `IS NOT NULL`，只要 WHERE 条件隐含被驱动表列非空即可。转为内连接后，优化器可以自由评估不同连接顺序的成本，选出最优方案。

子查询优化

子查询分类

按结果集分类：

类型	说明	示例

标量子查询	返回单一值	`WHERE m1 = (SELECT MIN(m2) FROM t2)`
行子查询	返回一行多列	`WHERE (m1,n1) = (SELECT m2,n2 FROM t2 LIMIT 1)`
列子查询	返回一列多行	`WHERE m1 IN (SELECT m2 FROM t2)`
表子查询	返回多行多列	`WHERE (m1,n1) IN (SELECT m2,n2 FROM t2)`

按关联性分类：

- 不相关子查询：可独立执行，不依赖外层查询
- 相关子查询：执行依赖外层查询的值，如 `WHERE n1 = n2` 中 n1 来自外层表

标量/行子查询执行方式

- 不相关：先执行子查询，结果作为常量代入外层查询（两个独立单表查询）
- 相关：逐行取外层记录 → 代入子查询执行 → 检查条件 → 循环

无特殊优化，执行方式符合直觉。

IN 子查询优化

IN 子查询是最常用的子查询类型，MySQL 投入了大量优化。

物化（Materialization）

问题：子查询结果集过大时，直接当 IN 参数会导致：内存放不下、无法用索引、逐条比对代价高。

方案：将子查询结果集写入临时表（物化表）：

- 自动去重（IN 不关心重复值）
- 小结果集 → Memory 引擎 + 哈希索引（查询极快）
- 大结果集（超过 `tmp_table_size`）→ 磁盘引擎 + B+ 树索引

物化表转连接：物化后，原查询等价于外层表与物化表的内连接：

```
-- 原始
SELECT * FROM s1 WHERE key1 IN (SELECT common_field FROM s2 WHERE key3 = 'a');
-- 物化后等价于
SELECT s1.* FROM s1 INNER JOIN materialized_table ON key1 = m_val;
```

转为连接后，优化器可以评估不同连接顺序的成本，选最优方案。

半连接（Semi-Join）

核心思想：将 IN 子查询直接转为连接，但只关心被驱动表中是否存在匹配记录，不关心有多少条。

```
-- 概念上等价于（MySQL 内部表示，非合法语法）
SELECT s1.* FROM s1 SEMI JOIN s2 ON s1.key1 = s2.common_field WHERE s2.key3 = 'a';
```

五种执行策略：

策略	原理	适用场景
Table Pullout	子查询表直接上拉到外层 FROM，转为普通内连接	子查询列是主键或唯一索引
DuplicateWeedout	建临时表（以外层表主键去重），消除重复结果	通用
LooseScan	扫描子查询表索引时，相同值只取第一条去匹配	子查询可用索引且查询列就是索引列
Materialization	先物化子查询再转连接（物化表天然去重）	不相关子查询
FirstMatch	逐行取外层记录，找到第一条匹配就停止	通用（最原始方式）

Semi-Join 适用条件：

- IN 子查询在 WHERE 或 ON 子句中，且与其他条件用 AND 连接
- 子查询是单一查询（非 UNION）
- 子查询不含 GROUP BY、HAVING、聚集函数

不能用 Semi-Join 的情况：

- IN 与其他条件用 OR 连接
- NOT IN
- IN 子查询在 SELECT 子句中
- 子查询含 GROUP BY / HAVING / 聚集函数 / UNION

不能 Semi-Join 时的后备策略

1. 物化后查询（仅不相关子查询）：物化子查询结果，但不转为连接，逐行判断值是否在物化表中
2. IN 转 EXISTS：转为 EXISTS 后可能用到子查询表的索引。仅在 WHERE/ON 中使用时 NULL 语义差异可忽略

```
-- 转换后可能用上 idx_key3 索引
WHERE EXISTS (SELECT 1 FROM s2 WHERE s1.common_field = s2.common_field AND s2.key3 = s1.key1)
```

IN 子查询优化决策总结

```
IN 子查询
├── 满足 semi-join 条件
│   └── 从 5 种策略中选成本最低的执行
└── 不满足 semi-join 条件
    ├── 不相关子查询 → 物化
    └── 相关子查询（或物化成本太高）→ IN 转 EXISTS
```

ANY/ALL 子查询优化

不相关的 ANY/ALL 子查询会转换为标量子查询：

原始表达式	转换为

`< ANY (SELECT ...)`	`< (SELECT MAX(...))`
`> ANY (SELECT ...)`	`> (SELECT MIN(...))`
`< ALL (SELECT ...)`	`< (SELECT MIN(...))`
`> ALL (SELECT ...)`	`> (SELECT MAX(...))`

EXISTS 子查询

- 不相关 EXISTS：先执行子查询得到 TRUE/FALSE，直接重写外层条件
- 相关 EXISTS：只能按逐行关联方式执行，但如果子查询能用索引则速度可观

派生表优化

FROM 子句中的子查询结果称为派生表（Derived Table）。两种优化策略：

合并到外层查询（优先）

将派生表展开，搜索条件合并到外层：

```
-- 原始
SELECT * FROM (SELECT * FROM s1 WHERE key1 = 'a') AS derived_s1
    INNER JOIN s2 ON derived_s1.key1 = s2.key1 WHERE s2.key2 = 1;
-- 合并后
SELECT * FROM s1 INNER JOIN s2 ON s1.key1 = s2.key1
    WHERE s1.key1 = 'a' AND s2.key2 = 1;
```

不能合并的情况：派生表含聚集函数、DISTINCT、GROUP BY、HAVING、LIMIT、UNION 或嵌套子查询。

物化

不能合并时，将派生表结果写入临时表。MySQL 采用延迟物化策略——只有真正用到派生表时才物化，避免不必要的开销。

决策顺序：优先尝试合并 → 不行则物化。

EXPLAIN 详解

在查询语句前加 `EXPLAIN`，MySQL 会输出该语句的执行计划，而非执行语句本身。执行计划展示了查询优化器选择的访问方法、使用的索引、预估扫描行数等关键信息。

基本用法

```
EXPLAIN SELECT * FROM s1 WHERE key1 = 'a';
```

输出包含 12 列：

列名	含义
`id`	每个 SELECT 对应一个唯一 id
`select_type`	查询类型（简单查询、子查询、UNION 等）
`table`	当前行对应的表名
`type`	访问方法（性能关键列）
`possible_keys`	可能用到的索引
`key`	实际使用的索引
`key_len`	使用索引的长度
`ref`	与索引列等值匹配的对象
`rows`	预估扫描行数
`filtered`	经过条件过滤后的百分比
`Extra`	额外信息

> `partitions` 列一般为 NULL，不常用，此处省略。

各列详解

id

每个 `SELECT` 关键字分配一个唯一的 `id`。

同一个 SELECT（含连接查询）：多表的 id 相同，排在前面的是驱动表，后面的是被驱动表：

```
EXPLAIN SELECT * FROM s1 INNER JOIN s2;
-- 两条记录 id 都是 1，s1 为驱动表，s2 为被驱动表
```

包含子查询：每个 SELECT 各自有不同的 id：

```
EXPLAIN SELECT * FROM s1 WHERE key1 IN (SELECT key1 FROM s2) OR key3 = 'a';
-- s1 对应 id=1（PRIMARY），s2 对应 id=2（SUBQUERY）
```

注意：查询优化器可能将子查询重写为连接查询，此时所有记录的 id 相同。如果执行计划中子查询的 id 和外层一样，说明优化器做了这个转换。

UNION 查询：每个 SELECT 各自有不同 id，`UNION`（需去重）还会出现 id 为 NULL 的临时表记录；`UNION ALL` 不去重，没有临时表。

select_type

值	含义
`SIMPLE`	简单查询，不含子查询或 UNION（连接查询也算 SIMPLE）
`PRIMARY`	包含 UNION/子查询的大查询中，最外层的查询
`UNION`	UNION 中第二个及之后的 SELECT
`UNION RESULT`	UNION 去重所用临时表
`SUBQUERY`	不相关子查询（会被物化，只执行一次）
`DEPENDENT SUBQUERY`	相关子查询（可能执行多次）
`DEPENDENT UNION`	UNION 中依赖外层查询的第二个及之后的 SELECT
`DERIVED`	派生表（FROM 子句中的子查询，物化执行）
`MATERIALIZED`	子查询被物化后与外层表做连接

table

每条执行计划记录对应一个表的访问方式。`table` 列显示表名，特殊情况：

- `<derivedN>`：id 为 N 的派生表（物化后的子查询）
- `<subqueryN>`：id 为 N 的物化子查询
- `<union1,2>`：UNION 去重用的临时表

type（访问类型）

性能从好到差排列，这是优化查询时最需要关注的列：

类型	含义	典型场景
`system`	表只有一行（MyISAM/Memory 引擎）	极少见
`const`	主键或唯一索引等值匹配	`WHERE id = 5`
`eq_ref`	连接查询中，被驱动表通过主键/唯一索引等值匹配	`JOIN ON s1.id = s2.id`
`ref`	普通二级索引等值匹配	`WHERE key1 = 'a'`
`ref_or_null`	与 ref 类似，额外搜索 NULL 值	`WHERE key1 = 'a' OR key1 IS NULL`
`index_merge`	使用多个索引合并（Intersection/Union/Sort-Union）	`WHERE key1 = 'a' OR key3 = 'a'`

`range`	索引范围扫描	`WHERE key1 > 'a'`、`WHERE key1 IN ('a','b')`
`index`	扫描整个索引树（覆盖索引但无法用范围/等值）	查询列和条件都在某个索引中，但条件不是最左前缀
`ALL`	全表扫描	无可用索引

核心口诀：至少要优化到 `range` 级别，`ref` 及以上更好。出现 `ALL` 应重点排查。

> `unique_subquery` 和 `index_subquery` 是 IN 子查询转 EXISTS 后的特殊类型，实际中较少见。

possible_keys 与 key

- `possible_keys`：查询中可能使用到的索引
- `key`：优化器最终选择的索引

possible_keys 不为空但 key 为 NULL，说明优化器经过成本计算后认为全表扫描更划算。

有一个特殊情况：`type = index` 时，possible_keys 可能为空而 key 不为空，因为此时虽然用了索引但不是通过正常的搜索条件命中的。

> possible_keys 越多，优化器计算成本的开销越大。应当删除不用的索引。

key_len

`key_len` 表示实际使用的索引长度（字节数），主要用于判断联合索引用到了几个列。

计算规则：

因素	说明
固定长度类型	INT = 4 字节，BIGINT = 8 字节
变长类型	VARCHAR(N) × 字符集字节数（utf8=3, utf8mb4=4）
允许 NULL	多 1 字节
变长字段	多 2 字节（存储实际长度）

示例（字符集 utf8）：

```
-- INT NOT NULL → key_len = 4
EXPLAIN SELECT * FROM s1 WHERE id = 5;              -- key_len: 4

-- INT（允许 NULL）→ key_len = 5
EXPLAIN SELECT * FROM s1 WHERE key2 = 5;             -- key_len: 5

-- VARCHAR(100)（允许 NULL）→ 100×3 + 1 + 2 = 303
EXPLAIN SELECT * FROM s1 WHERE key1 = 'a';           -- key_len: 303
```

判断联合索引使用了几列：

```sql
-- idx_key_part(key_part1, key_part2, key_part3), 每列 VARCHAR(100)

EXPLAIN SELECT * FROM s1 WHERE key_part1 = 'a';
-- key_len: 303 → 只用了 1 列

EXPLAIN SELECT * FROM s1 WHERE key_part1 = 'a' AND key_part2 = 'b';
-- key_len: 606 → 用了 2 列
```

ref

当 type 为 `const`/`eq_ref`/`ref`/`ref_or_null` 等等值匹配时，ref 列显示与索引列匹配的对象：

- `const`：与常量匹配（如 `WHERE key1 = 'a'`）
- `db.table.column`：与某表的某列匹配（如连接条件 `ON s1.id = s2.id`）
- `func`：与函数返回值匹配（如 `ON s2.key1 = UPPER(s1.key1)`）

rows 与 filtered

rows：预估需要扫描的行数。全表扫描时就是表的总行数估算，使用索引时是满足索引条件的估算行数。

filtered：在 rows 条记录中，满足其余搜索条件的百分比。

对于连接查询，驱动表的扇出 = rows × filtered%，决定了被驱动表被访问的次数：

```sql
EXPLAIN SELECT * FROM s1 INNER JOIN s2 ON s1.key1 = s2.key1 WHERE s1.common_field = 'a';
-- s1: rows=9688, filtered=10.00 → 扇出 = 968.8
-- 意味着 s2 会被查询约 968 次
```

Extra

Extra 列提供额外的执行信息，以下是最常见且重要的值：

Using index（覆盖索引）

查询的列和条件都在索引中，不需要回表。性能好的标志。

```sql
EXPLAIN SELECT key1 FROM s1 WHERE key1 = 'a';
-- Extra: Using index
```

Using where

- 全表扫描 + WHERE 过滤条件
- 使用索引后，还有额外的非索引条件需要过滤

```sql
EXPLAIN SELECT * FROM s1 WHERE common_field = 'a';
-- type: ALL, Extra: Using where

EXPLAIN SELECT * FROM s1 WHERE key1 = 'a' AND common_field = 'a';
-- type: ref, Extra: Using where（key1 走索引，common_field 在服务层过滤）
```

Using index condition（索引条件下推 ICP）

索引列上的部分条件无法用于索引范围扫描，但可以在索引层面先过滤，减少回表次数。

```
-- key1 > 'z' 走索引范围扫描，key1 LIKE '%b' 在索引层下推过滤
EXPLAIN SELECT * FROM s1 WHERE key1 > 'z' AND key1 LIKE '%b';
-- Extra: Using index condition
```

`Using filesort（文件排序）`

排序无法利用索引，必须在内存或磁盘中完成排序。性能警告，应尽量优化为索引排序。

```
EXPLAIN SELECT * FROM s1 ORDER BY common_field LIMIT 10;
-- Extra: Using filesort
```

`Using temporary（临时表）`

执行 DISTINCT、GROUP BY、UNION
等操作时无法利用索引，创建了内部临时表。性能警告，应尽量优化。

```
EXPLAIN SELECT DISTINCT common_field FROM s1;
-- Extra: Using temporary

EXPLAIN SELECT common_field, COUNT(*) FROM s1 GROUP BY common_field;
-- Extra: Using temporary; Using filesort
-- GROUP BY 默认会加 ORDER BY，可追加 ORDER BY NULL 去掉排序
```

用索引列做 GROUP BY 就不需要临时表：

```
EXPLAIN SELECT key1, COUNT(*) FROM s1 GROUP BY key1;
-- Extra: Using index（直接扫描索引即可）
```

`Using join buffer (Block Nested Loop)`

连接查询中被驱动表无法有效利用索引，使用了 join buffer 加速。说明被驱动表的连接条件没有索引。

`Using intersect/union/sort_union`

使用了索引合并策略，括号内显示合并的索引名。

其他值：

值	含义
`No tables used`	查询没有 FROM 子句
`Impossible WHERE`	WHERE 条件恒为 FALSE
`Not exists`	左连接中被驱动表条件为 IS NULL，找到一条匹配即可
`Start/End temporary`	semi-join 的 DuplicateWeedout 策略
`LooseScan`	semi-join 的 LooseScan 策略
`FirstMatch(tbl)`	semi-join 的 FirstMatch 策略

输出格式

除了默认的表格格式，还可以使用 JSON 格式查看更详细的成本信息：

```
EXPLAIN FORMAT=JSON SELECT * FROM s1 INNER JOIN s2 ON s1.key1 = s2.key2\G
```

JSON 格式额外提供：

- `cost_info.prefix_cost`：查询到当前表为止的总成本
- `rows_examined_per_scan`：每次扫描的行数
- `rows_produced_per_join`：扇出行数

使用 `EXPLAIN` 后紧接着执行 `SHOW WARNINGS`，可以查看优化器重写后的查询语句（Code=1003），帮助理解优化器的改写行为（如左连接优化为内连接）。

第三部分：事务与锁

事务简介

事务的概念

事务（transaction）是一个或多个数据库操作的逻辑单元，这些操作要么全部成功，要么全部失败。事务必须满足 ACID 四个特性。

典型场景：转账操作包含两条 UPDATE
语句（一条扣钱、一条加钱），它们必须作为一个整体执行，不能只完成一半。

ACID 特性

原子性（Atomicity）

事务中的所有操作要么全部执行成功，要么全部回滚到执行前的状态，不存在"执行了一半"的中间态。

数据库通过 undo log 实现原子性——执行过程中发生错误时，利用 undo log 把已完成的操作撤销。

一致性（Consistency）

事务执行前后，数据库都必须满足所有既定约束（主键、唯一索引、外键、NOT NULL
以及业务规则等）。

一致性由两方面保证：

- 数据库层面：主键约束、唯一索引、外键、NOT NULL 等。MySQL 支持 `CHECK`
  语法但不实际执行检查（SQL Server/Oracle 会执行）；可用触发器自定义约束。
- 应用层面：复杂业务规则（如余额不能为负）需要程序员在代码中保证。

原子性和隔离性是保证一致性的手段，一致性是最终结果。满足原子性和隔离性不一定满足一致性（如余额变负），反之亦然。

隔离性（Isolation）

并发执行的多个事务之间互不干扰。若不做隔离控制，并发事务的操作可能交叉执行，导致数据不一致（如两次转账只扣了一次钱却加了两次钱）。

数据库通过锁机制和 MVCC 来保证不同隔离级别下的隔离性。

持久性（Durability）

事务一旦提交，其对数据库的修改就永久保存到磁盘上，即使之后系统崩溃也不会丢失。

数据库通过 redo log 实现持久性——数据修改先写 redo log，确保崩溃后可重做。

事务的状态

事务从开始到结束经历以下状态：

状态	含义
活动的（active）	事务正在执行中
部分提交的（partially committed）	最后一个操作执行完成，但修改还在内存中，尚未刷盘
失败的（failed）	在 active 或 partially committed 阶段遇到错误，无法继续执行
中止的（aborted）	失败事务完成回滚，数据库恢复到事务执行前的状态
提交的（committed）	部分提交的事务成功将修改刷到磁盘，事务完成

状态转换路径：

```
active → partially committed → committed
  ↓              ↓
failed    →    failed    →    aborted
```

事务生命周期在 `committed` 或 `aborted` 状态结束。

MySQL 中的事务用法

MySQL 中只有 InnoDB（和 NDB）支持事务。MyISAM 不支持事务，ROLLBACK 对其无效。

开启事务（BEGIN / START TRANSACTION）

```
-- 方式一
BEGIN;

-- 方式二（支持修饰符）
START TRANSACTION;
START TRANSACTION READ ONLY;                -- 只读事务
START TRANSACTION READ WRITE;               -- 读写事务（默认）
START TRANSACTION WITH CONSISTENT SNAPSHOT; -- 一致性读
START TRANSACTION READ ONLY, WITH CONSISTENT SNAPSHOT; -- 可组合
```

`READ ONLY` 和 `READ WRITE` 不能同时使用。只读事务仍可操作临时表。

提交与回滚

```
COMMIT;     -- 提交事务
ROLLBACK;   -- 回滚事务，撤销所有修改
```

执行过程中遇到错误时，事务会自动回滚。手动发现问题也可主动 ROLLBACK。

保存点（SAVEPOINT）

保存点允许回滚到事务中的某个中间位置，而非回到起点。

```
SAVEPOINT s1;                -- 创建保存点
ROLLBACK TO s1;              -- 回滚到保存点 s1（s1 之后的操作撤销）
RELEASE SAVEPOINT s1;        -- 删除保存点
```

不带保存点名称的 `ROLLBACK` 回滚整个事务。

autocommit

```
SHOW VARIABLES LIKE 'autocommit';  -- 默认 ON
```

autocommit = ON 时，每条语句自动作为独立事务提交。关闭自动提交的方式：

1. 用 `BEGIN` / `START TRANSACTION` 显式开启事务（提交/回滚后恢复自动提交）
2. `SET autocommit = OFF;`（后续语句属于同一事务，直到显式 COMMIT 或 ROLLBACK）

隐式提交

某些语句会自动提交当前未完成的事务：

- DDL 语句：`CREATE` / `ALTER` / `DROP` 表、视图、存储过程等
- 用户管理：`CREATE USER` / `ALTER USER` / `GRANT` / `REVOKE` 等
- 事务控制：未提交时再次 `BEGIN`，或将 autocommit 从 OFF 改为 ON
- 锁语句：`LOCK TABLES` / `UNLOCK TABLES`
- 数据加载：`LOAD DATA`
- 复制命令：`START SLAVE` / `STOP SLAVE` 等
- 维护语句：`ANALYZE TABLE` / `OPTIMIZE TABLE` / `CHECK TABLE` 等

redo 日志

为什么需要 redo 日志

InnoDB 以页（16KB）为单位管理存储，所有修改先在内存 Buffer Pool 中进行。事务提交后如果系统崩溃，内存中的修改就会丢失，无法满足事务的持久性（Durability）。

最直觉的方案是事务提交时把所有修改过的页刷到磁盘，但有两个致命问题：

1. 写放大：可能只改了页中几个字节，却要刷整个 16KB 页
2. 随机 IO：一个事务可能修改多个不相邻的页，产生大量随机写

解决思路就是 WAL（Write-Ahead Logging）：不刷数据页，而是把"做了什么修改"以日志形式顺序写到磁盘。这就是 redo 日志。

redo 日志的两大优势：

- 体积小：只记录"哪个表空间、哪个页、哪个偏移量、改成什么值"
- 顺序写：日志按产生顺序追加写入，顺序 IO 远快于随机 IO

崩溃重启时，按 redo 日志重放修改即可恢复数据，保证持久性。

redo 日志格式

每条 redo 日志的通用结构：

字段	含义
type	日志类型（共 53 种）
space ID	表空间 ID
page number	页号
data	具体修改内容

简单类型（物理日志）

直接记录"在某页某偏移量处写入 N 个字节"，包括：

- `MLOG_1BYTE` / `MLOG_2BYTE` / `MLOG_4BYTE` / `MLOG_8BYTE`：写入固定字节数
- `MLOG_WRITE_STRING`：写入变长数据，额外带 `len` 字段

这类日志结构为：`type + space ID + page number + offset + data`。

复杂类型（逻辑日志）

一条 INSERT 可能修改页面中的多个位置（记录数据、Page Directory、Page Header、记录链表等）。如果每个修改点都写一条物理日志，开销太大。

因此 InnoDB 设计了复合类型日志，如 `MLOG_COMP_REC_INSERT`（插入紧凑行格式记录）、`MLOG_COMP_REC_DELETE`（删除记录）、`MLOG_COMP_PAGE_CREATE`（创建页面）等。

这类日志的特点：

- 物理层面：指明了对哪个表空间的哪个页修改
- 逻辑层面：恢复时不是简单地覆盖字节，而是调用对应函数（如"插入记录"函数），用日志中的数据作为参数重新执行操作，间接恢复页面所有相关字段

Mini-Transaction（mtr）

为什么需要 mtr

向 B+ 树插入一条记录时，可能涉及页分裂，需要修改多个页面（原页、新页、父节点页、段/区管理页等），产生几十条 redo
日志。这些修改必须是原子的——要么全部恢复，要么全部不恢复，否则会产生不正确的 B+ 树结构。

mtr 的原子性保证

InnoDB 把对底层页面的一次原子操作称为一个 Mini-Transaction（mtr）。一个 mtr 产生的一组 redo 日志是不可分割的整体。

实现方式：

- 多条日志的 mtr：在最后一条日志后追加一条 `MLOG_MULTI_REC_END`
  类型的标记日志。恢复时只有扫描到该标记才认为是完整的一组，否则丢弃
- 单条日志的 mtr：利用 type 字段的最高位（第 1 bit）标记为 1，表示该条日志本身就是一个完整的 mtr

事务、语句与 mtr 的关系

```
事务 → 包含多条语句
  语句 → 包含多个 mtr
    mtr → 包含一组 redo 日志（不可分割）
```

redo log buffer

基本结构

InnoDB 启动时申请一块连续内存作为 redo log buffer，由参数 `innodb_log_buffer_size`
控制大小（默认 16MB）。

log buffer 内部划分为若干个 512 字节的 block（称为 redo log block）。每个 block 结构：

部分	大小	说明
log block header	12 字节	管理信息（block 编号、已用字节数等）
log block body	496 字节	实际存储 redo 日志
log block trailer	4 字节	校验值

写入时通过全局变量 `buf_free` 标记下一条日志的写入位置，顺序追加。

写入时机

mtr 执行过程中产生的日志先暂存于 mtr 内部，`mtr` 结束时才将整组日志一次性复制到 `log buffer`。这保证了同一 mtr 的日志在 buffer 中是连续的。

不同事务的 mtr 可能并发执行，因此 log buffer 中不同事务的日志是交替存放的。

刷盘时机

log buffer 中的日志在以下情况刷入磁盘：

1. `log buffer` 空间不足：已用空间超过总容量一半时触发刷盘
2. 事务提交时：根据 `innodb_flush_log_at_trx_commit` 参数决定行为
3. 后台线程：约每秒刷新一次
4. 做 `checkpoint` 时
5. 服务器正常关闭时

刷盘策略：`innodb_flush_log_at_trx_commit`

值	行为	持久性	性能
0	提交时不刷盘，交给后台线程	崩溃可能丢最近约 1 秒的事务	最快
1	提交时同步刷盘（fsync）	完全保证持久性	最慢（默认值）
2	提交时写到 OS 缓冲区，不 fsync	数据库崩溃不丢，OS 崩溃可能丢	折中

生产环境通常使用 1；对允许少量丢失的场景（如日志表）可用 2 提升性能。

`redo` 日志文件

文件组结构

MySQL 数据目录下默认有 `ib_logfile0` 和 `ib_logfile1` 两个日志文件，构成一个日志文件组。相关参数：

参数	含义	默认值
`innodb_log_file_size`	单个日志文件大小	48MB
`innodb_log_files_in_group`	日志文件个数	2
`innodb_log_group_home_dir`	日志文件目录	数据目录

总日志容量 = `innodb_log_file_size` × `innodb_log_files_in_group`。

循环写入

日志文件组采用循环写：写满 `ib_logfile0` → 写 `ib_logfile1` → ⋯ → 写满最后一个文件后回到 `ib_logfile0`。每个文件前 2048 字节（4 个 block）存储管理信息（包括 checkpoint 信息），实际日志从偏移量 2048 开始。

循环写意味着新日志会覆盖旧日志，因此需要 checkpoint 机制来标记哪些旧日志可以安全覆盖。

LSN（Log Sequence Number）

LSN 是一个全局递增的日志序列号，表示系统已产生的 redo 日志总量（单位：字节）。初始值为 8704。

每个 mtr 写入 log buffer 时，lsn 增加该 mtr 日志的字节数（跨 block 时还要加上 block header/trailer 的开销）。LSN 越小，日志越早产生。

几个关键 LSN

变量	含义
`lsn`	当前系统已写入（含 log buffer 中）的日志总量
`flushed_to_disk_lsn`	已刷到磁盘的日志量。等于 lsn 时表示所有日志已落盘
`checkpoint_lsn`	已做 checkpoint 的位置，之前的日志可被覆盖

可通过 `SHOW ENGINE INNODB STATUS` 查看：

```
Log sequence number 124476971    -- lsn
Log flushed up to   124099769    -- flushed_to_disk_lsn
Pages flushed up to 124052503    -- flush 链表尾部 oldest_modification
Last checkpoint at  124052494    -- checkpoint_lsn
```

flush 链表与 LSN

mtr 结束时，除了将 redo 日志写入 log buffer，还会把修改过的页加入 Buffer Pool 的 flush 链表（按首次修改时间从新到旧排序）。每个脏页维护两个 LSN：

- `oldest_modification`：该页首次被修改时 mtr 的起始 lsn
- `newest_modification`：该页最近一次被修改时 mtr 的结束 lsn

checkpoint

为什么需要 checkpoint

日志文件组空间有限，循环写会覆盖旧日志。只有当 redo 日志对应的脏页已经刷到磁盘，该段日志才不再需要，可以被覆盖。

checkpoint 过程

1. 计算 checkpoint_lsn：取 flush 链表尾部（最早修改的脏页）的 `oldest_modification` 值，所有 lsn 小于该值的日志均可覆盖
2. 持久化 checkpoint 信息：将 `checkpoint_lsn`、对应的日志文件偏移量、checkpoint 编号（`checkpoint_no`）写入第一个日志文件的管理区域（`checkpoint_no` 为偶数写 checkpoint1 block，奇数写 checkpoint2 block）

write pos 与 checkpoint 的追赶关系

可以把循环日志想象为一个环形结构：

- `write pos`：当前日志写入位置，不断前进
- `checkpoint`：可覆盖位置，随脏页刷盘而前进

write pos 追上 checkpoint 时，意味着日志空间满了，必须先推进 checkpoint（刷脏页）才能继续写入。如果后台线程刷脏不够快，用户线程会被迫同步刷脏。

崩溃恢复

确定恢复起点

读取第一个日志文件中 checkpoint1 和 checkpoint2 的 `checkpoint_no`，取较大的那个。其对应的 `checkpoint_lsn` 和 `checkpoint_offset` 就是恢复的起点——之前的日志对应的脏页已持久化，无需恢复。

确定恢复终点

从起点开始顺序扫描 block。每个 block 的 `LOG_BLOCK_HDR_DATA_LEN` 记录已使用字节数，被写满的 block 该值为 512。第一个该值不为 `512` 的 `block` 就是终点。

恢复过程

1. 扫描 `redo` 日志：从 checkpoint_lsn 对应的偏移量开始，顺序读取到终点
2. 按页面分组（哈希表优化）：以 `(space ID, page number)` 为 key 建哈希表，同一页的 redo 日志按生成顺序串成链表。这样可以一次性读取一个页并应用所有相关日志，将随机 IO 转为顺序处理
3. 应用日志恢复页面：遍历哈希表，逐页应用 redo 日志
4. 跳过已刷盘页面：每个数据页的 File Header 中有 `FIL_PAGE_LSN` 属性（等于该页的 `newest_modification`）。若该值 ≥ 某条 redo 日志的 lsn，说明该修改已经持久化，无需重复应用，进一步加速恢复

undo 日志

为什么需要 undo 日志

事务要求原子性：要么全做，要么全不做。如果事务执行到一半遇到错误或手动 `ROLLBACK`，就需要把已做的修改全部撤销——这就是回滚。

回滚的本质是"反向操作"：

- 插入了一条记录 → 回滚时删除它
- 删除了一条记录 → 回滚时重新插入
- 修改了一条记录 → 回滚时还原旧值

每次对记录做增删改之前，InnoDB 会先记录一条 undo 日志，保存回滚所需的信息。SELECT 不修改数据，不产生 undo 日志。

除了回滚之外，undo 日志的另一个核心用途是支撑 MVCC（多版本并发控制）。通过 undo 日志中记录的历史版本，其他事务可以读到某条记录"在过去某个时间点"的快照版本，而不必等待当前事务提交。

事务 ID

分配时机

InnoDB 不会在事务开启时立即分配事务 ID，而是：

- 读写事务：在它第一次对某个表执行增删改时，才分配一个唯一的 `trx_id`
- 只读事务：只有对用户创建的临时表做增删改时才分配
- 如果一个事务全是 SELECT，不会分配事务 ID

生成方式

服务器在内存中维护一个全局递增变量，每次分配事务 ID 就取当前值并自增。每当该值为 256 的倍数时，持久化到系统表空间（第 5 号页面的 `Max Trx ID`）。重启后加载该值 +256 继续分配，保证全局单调递增。

trx_id 和 roll_pointer 隐藏列

聚簇索引的每条记录自动携带两个隐藏列：

- `trx_id`：最近一次对该记录做改动的事务 ID
- `roll_pointer`：指向该记录对应的 undo 日志的指针（7 字节），是构建版本链的关键

INSERT 的 undo 日志

类型：TRX_UNDO_INSERT_REC

插入操作的回滚只需要"按主键删除"，因此 undo 日志只记录：

- undo no：本事务中第几条 undo 日志（从 0 递增）
- table id：表的 ID
- 主键各列的长度和值

回滚时根据主键值执行删除，聚簇索引记录被删后，对应的二级索引记录也会被一并清除。

插入记录后，该记录的 roll_pointer 就指向这条 undo 日志。

DELETE 的 undo 日志

删除操作分两个阶段：

阶段一：delete mark

在事务提交前，只将记录头信息中的 delete_mask 标志位置为 1，记录仍留在正常记录链表中，不会真正移除。同时更新 trx_id 和 roll_pointer。

此时记录处于"中间状态"——标记了删除但物理上还在。这是为 MVCC 服务的：其他事务可能还需要读取这条记录的内容。

这一步产生的 undo 日志类型为 TRX_UNDO_DEL_MARK_REC，记录内容包括：

- undo no、table id
- old trx_id、old roll_pointer（记录修改前的值，用于串联版本链）
- 索引列各列信息（<pos, len, value> 列表），供 purge 阶段使用

阶段二：purge

事务提交后，由后台 purge 线程异步执行真正的物理删除：将记录从正常链表移到垃圾链表，释放存储空间供后续复用。

undo 日志只需要记录阶段一（delete mark）的信息，因为提交后不再需要回滚。

UPDATE 的 undo 日志

UPDATE 操作根据是否更新主键，处理方式完全不同。

不更新主键

分两种情况：

就地更新（in-place update）：被更新的每个列更新前后占用的存储空间大小都相同，就直接在原位置修改。例如 VARCHAR 列从 'M416'（4 字节）更新为 'M249'（4 字节）。任何一个列的大小变了都不能就地更新。

先删后插：如果有列大小发生变化，先把旧记录从聚簇索引页面中真正删除（移入垃圾链表，注意这里不是 delete mark，是用户线程同步执行的真正删除），然后插入一条用更新后列值构建的新记录。

两种情况产生的 undo 日志类型相同：TRX_UNDO_UPD_EXIST_REC，记录内容包括：

- undo no、table id
- old trx_id、old roll_pointer

- `n_updated`：被更新的列数
- 每个被更新列的 `<pos, old_len, old_value>`（旧值信息）
- 如果更新了索引列，还会额外记录索引列各列信息

更新主键

更新主键意味着记录在聚簇索引中的位置要改变，InnoDB 分两步处理：

1. 对旧记录执行 `delete mark`（不是真删，purge 线程后续处理）→ 产生一条 TRX_UNDO_DEL_MARK_REC 类型的 undo 日志
2. 根据新主键值插入新记录（重新定位位置）→ 产生一条 TRX_UNDO_INSERT_REC 类型的 undo 日志

所以更新主键时，一条记录的修改会产生 2 条 undo 日志。

旧记录只做 delete mark 而不立即真删，是因为其他事务可能正在通过 MVCC 读取该版本。

版本链

每条 undo 日志中都保存了 `old trx_id` 和 `old roll_pointer`。记录的 `roll_pointer` 指向最新的 undo 日志，而这条 undo 日志通过 `old roll_pointer` 又指向更早的 undo 日志，如此串联形成一条版本链。

```
当前记录 (trx_id=200)
   ↓ roll_pointer
undo日志 (trx_id=200, old_roll_pointer →)
   ↓
undo日志 (trx_id=100, old_roll_pointer →)
   ↓
undo日志 (trx_id=50, ...)
```

通过版本链，MVCC 可以沿着链表回溯，找到对某个事务"可见"的那个历史版本，实现不加锁的一致性读。这是下一章 MVCC 的核心基础。

undo 日志的存储

undo 日志的两大类

InnoDB 将 undo 日志分为两大类，分开存储在不同的 Undo 页面中，不能混放：

类别	包含的日志类型	说明
TRX_UNDO_INSERT	TRX_UNDO_INSERT_REC	INSERT 产生（含 UPDATE 更新主键时的插入）
TRX_UNDO_UPDATE	TRX_UNDO_DEL_MARK_REC、TRX_UNDO_UPD_EXIST_REC 等	DELETE、UPDATE 产生

分开的原因：INSERT 类的 undo 日志提交后可立即释放；UPDATE 类因 MVCC 依赖不能立即释放。

Undo 页面链表

一个事务执行中，undo 日志写在类型为 `FIL_PAGE_UNDO_LOG` 的页面中。一个页面放不下时追加新页面，通过链表串联。由于两大类不能混放，一个事务最多产生 4 条 Undo 页面链表：

- 普通表的 insert undo 链表
- 普通表的 update undo 链表
- 临时表的 insert undo 链表
- 临时表的 update undo 链表

按需分配，不需要就不分配。不同事务的 undo 日志写入不同的链表，互不干扰。

回滚段（Rollback Segment）

每条 Undo 页面链表需要占用一个 undo slot。InnoDB 用一个 `Rollback Segment Header` 页面来集中管理 undo slot，每个这样的页面包含 1024 个 undo slot。

InnoDB 共有 128 个回滚段，即 128 × 1024 = 131072 个 undo slot。回滚段分布如下：

编号	用途	位置
第 0 号	普通表	系统表空间
第 1~32 号	临时表	临时表空间（ibtmp1）
第 33~127 号	普通表	系统表空间或 undo 表空间

分配 undo slot 的流程：

1. 轮询（round-robin）选一个回滚段
2. 先看 cached 链表有无可重用的 slot
3. 没有则遍历该回滚段中 1024 个 slot，找到空闲的（值为 `FIL_NULL`）
4. 全满则报错 `Too many active concurrent transactions`

undo 表空间

第 33~127 号回滚段可配置到独立的 undo 表空间中（`innodb_undo_tablespaces`）。好处是 undo 表空间可以 truncate 缩小文件，而系统表空间只能增长不能缩小。

undo 日志的生命周期

insert undo 日志

事务提交后，INSERT 的 undo 日志就没用了（新插入的记录对其他事务不存在历史版本问题），因此可以立即释放。

如果 insert undo 链表只有一个页面且已用空间 < 3/4，则该链表会被缓存起来（标记为 `TRX_UNDO_CACHED`），后续事务可以直接覆盖重写，避免反复分配页面。

不满足重用条件的链表，对应的段直接释放，undo slot 置为 `FIL_NULL`。

update undo 日志

事务提交后，DELETE/UPDATE 的 undo 日志不能立即释放，因为其他事务可能正通过 MVCC 的版本链读取这些历史版本。

处理方式：

1. 事务提交时，将这组 undo 日志放入 `History` 链表
2. 后台 `purge` 线程定期检查：当没有任何活跃事务再需要某个历史版本时，才清理对应的 undo 日志并释放空间

如果 update undo 链表符合重用条件（单页面 + 已用 < 3/4），后续事务可以在同一页面中追加新的 undo 日志组（不能覆盖，因为旧的还有用）。

总结

类型	提交后处理	原因
insert undo	立即释放或缓存重用	新记录无历史版本需求
update undo	放入 History 链表，等 purge 清理	MVCC 版本链依赖

事务隔离级别与 MVCC

并发事务的问题

多个事务并发访问相同数据时，如果不加控制，可能产生以下四类问题（按严重程度排序）：

脏写（Dirty Write）

一个事务修改了另一个未提交事务修改过的数据。

事务 B 更新了某行，事务 A 又更新了同一行。如果事务 B 回滚，事务 A 的更新也一起消失。脏写是最严重的问题，任何隔离级别都不允许发生。InnoDB 通过锁机制保证：第一个事务更新记录后加锁，第二个事务必须等锁释放才能更新。

脏读（Dirty Read）

一个事务读到了另一个未提交事务修改过的数据。

事务 B 把 name 改成 `''`，事务 A 读到了 `''`，随后事务 B 回滚——事务 A 读到的数据实际上从未存在过。

不可重复读（Non-Repeatable Read）

一个事务内两次读取同一行，结果不同——因为另一个已提交事务在两次读取之间修改了该行。

事务 A 第一次读 name 为 `''`，事务 B 把 name 改成 `''` 并提交，事务 A 第二次读 name 变成了 `''`。

幻读（Phantom Read）

一个事务按相同条件两次查询，第二次读到了第一次没有的新行——因为另一个已提交事务在两次查询之间插入了符合条件的记录。

事务 A 查询 `number > 0`，得到 1 条记录。事务 B 插入一条新记录并提交。事务 A 再查 `number > 0`，得到 2 条记录。

幻读专指"多出来的行"。如果是已有行被删除导致变少，本质上属于每一行的不可重复读。

SQL 标准的四个隔离级别

隔离级别	脏读	不可重复读	幻读
READ UNCOMMITTED	可能	可能	可能
READ COMMITTED	不可能	可能	可能
REPEATABLE READ	不可能	不可能	可能
SERIALIZABLE	不可能	不可能	不可能

- 隔离级别越高，并发问题越少，但性能开销越大

- 脏写太严重，四个级别都不允许
- MySQL 默认隔离级别：REPEATABLE READ
- MySQL 的 RR 级别在实现上可以通过 MVCC + 间隙锁禁止幻读，比 SQL 标准更强

MySQL 中设置隔离级别

```
-- 语法
SET [GLOBAL|SESSION] TRANSACTION ISOLATION LEVEL level;

-- level 可选值
-- REPEATABLE READ | READ COMMITTED | READ UNCOMMITTED | SERIALIZABLE
```

作用范围：

关键字	影响范围
GLOBAL	对之后新建的会话生效，已有会话不受影响
SESSION	对当前会话的所有后续事务生效
都不写	仅对当前会话的下一个事务生效

```
-- 查看当前隔离级别
SELECT @@transaction_isolation;
-- 结果: REPEATABLE-READ

-- 启动参数修改默认级别
-- --transaction-isolation=SERIALIZABLE
```

MVCC 原理

MVCC（Multi-Version Concurrency Control，多版本并发控制）让读-写操作可以并发执行而不互相阻塞，是 RC 和 RR 两个隔离级别的核心实现机制。

版本链

InnoDB 聚簇索引的每条记录包含两个隐藏列：

隐藏列	作用
`trx_id`	最近一次修改该记录的事务 ID
`roll_pointer`	指向该记录修改前的 undo 日志（旧版本）

每次 UPDATE 都将旧值写入 undo 日志，通过 `roll_pointer` 将所有历史版本串成一条版本链，链头是最新版本。每个版本都带有生成它的 `trx_id`。

示例：初始记录由事务 80 插入 `(1, '', '')`，之后事务 100 先后更新为 `''`、`''`，版本链为：

```
[张飞, trx_id=100] → [关羽, trx_id=100] → [刘备, trx_id=80]
```

ReadView

ReadView 是事务在执行 SELECT 时生成的一致性快照，用于判断版本链中哪个版本对当前事务可见。

ReadView 包含 4 个关键字段：

字段	含义
`m_ids`	生成 ReadView 时，系统中所有活跃（未提交）的读写事务 ID 列表
`min_trx_id`	m_ids 中的最小值
`max_trx_id`	系统应该分配给下一个事务的 ID（不是 m_ids 的最大值）
`creator_trx_id`	生成该 ReadView 的事务自身的 ID（只读事务默认为 0）

> `max_trx_id` 的值 = 当前已分配的最大事务 ID + 1。例如活跃事务为 1、2（事务 3 已提交），则 m_ids=[1,2]，min_trx_id=1，max_trx_id=4。

可见性判断规则

访问版本链中某个版本时，取该版本的 `trx_id`，依次判断：

1. `trx_id == creator_trx_id` → 可见（自己修改的，当然看得到）
2. `trx_id < min_trx_id` → 可见（该版本在 ReadView 生成前已提交）
3. `trx_id >= max_trx_id` → 不可见（该版本由 ReadView 生成后才开启的事务产生）
4. `min_trx_id <= trx_id < max_trx_id` → 检查 trx_id 是否在 m_ids 中：- 在 m_ids 中 → 不可见（生成该版本的事务仍活跃）- 不在 m_ids 中 → 可见（生成该版本的事务已提交）

如果当前版本不可见，沿 `roll_pointer` 找到上一个版本，重复上述判断，直到找到可见版本或遍历完所有版本（该记录对当前事务完全不可见）。

READ COMMITTED 的 MVCC 实现

RC 级别下，每次执行 SELECT 都会生成一个新的 ReadView。

示例，初始数据 `(1, '', '')` 由事务 80 插入：

第一次 SELECT（事务 100 和 200 均未提交）：

- 事务 100 已将 name 更新为 `''` → `''`
- 生成 ReadView：m_ids=[100,200]，min_trx_id=100，max_trx_id=201
- 版本链：`(100)` → `(100)` → `(80)`
- 100 在 m_ids 中 → 不可见；80 < 100 → 可见
- 结果：`''`

第二次 SELECT（事务 100 已提交，事务 200 更新为 `''` → `''`）：

- 重新生成 ReadView：m_ids=[200]，min_trx_id=200，max_trx_id=201
- 版本链：`(200)` → `(200)` → `(100)` → `(100)` → `(80)`

- 200 在 m_ids 中 → 不可见；100 < 200 → 可见
- 结果：`''`（能看到事务 100 提交后的值）

两次读取结果不同——这就是不可重复读。

REPEATABLE READ 的 MVCC 实现

RR 级别下，只在第一次执行 SELECT 时生成 ReadView，之后复用同一个。

同样的场景：

第一次 SELECT（事务 100 和 200 均未提交）：

- 生成 ReadView：m_ids=[100,200]，min_trx_id=100，max_trx_id=201
- 判断过程同上，结果：`''`

第二次 SELECT（事务 100 已提交，事务 200 更新为 `''`）：

- 复用第一次的 ReadView：m_ids=[100,200]，min_trx_id=100，max_trx_id=201
- 版本链：`(200)` → `(200)` → `(100)` → `(100)` → `(80)`
- 200 在 m_ids → 不可见；100 在 m_ids → 不可见；80 < 100 → 可见
- 结果：仍然是 `''`

两次读取结果一致——这就是可重复读。

核心区别总结：

隔离级别	ReadView 生成时机	效果
READ COMMITTED	每次 SELECT 都生成新的	能看到其他已提交事务的修改
REPEATABLE READ	第一次 SELECT 时生成，之后复用	整个事务期间看到的数据快照一致

当前读与快照读

MVCC 中的读操作分两类：

快照读（Snapshot Read）：普通 SELECT 语句，读取的是版本链中符合 ReadView 可见性规则的历史版本，不加锁。

```
SELECT * FROM hero WHERE number = 1;
```

当前读（Current Read）：读取记录的最新已提交版本，并对记录加锁。以下操作都是当前读：

```
SELECT * FROM hero WHERE number = 1 FOR UPDATE;         -- 加排他锁
SELECT * FROM hero WHERE number = 1 LOCK IN SHARE MODE; -- 加共享锁
INSERT INTO hero VALUES (2, '关羽', '蜀');               -- 插入前检查
UPDATE hero SET name = '张飞' WHERE number = 1;          -- 更新
DELETE FROM hero WHERE number = 1;                       -- 删除
```

INSERT / UPDATE / DELETE 内部需要先读取最新数据再执行修改，因此始终是当前读，不走 MVCC 版本链。

幻读的解决

SQL 标准中 RR 级别仍可能幻读，但 MySQL 的 RR 级别通过两种机制配合解决幻读：

1. 快照读：MVCC 天然防幻读

RR 下 ReadView 只生成一次。即使其他事务插入了新行并提交，由于新行的 trx_id 要么在 m_ids 中、要么 >= max_trx_id，对当前事务不可见。快照读层面不会出现幻行。

2. 当前读：间隙锁（Gap Lock）防幻读

当执行 `SELECT ... FOR UPDATE` 或 UPDATE/DELETE 等当前读操作时，InnoDB 不仅锁住已有记录（Record Lock），还会锁住记录之间的间隙（Gap Lock），两者合称 Next-Key Lock。

间隙锁阻止其他事务在锁定范围内插入新记录，从而在当前读层面也避免了幻读。

> 间隙锁只在 RR 级别下生效。RC 级别没有间隙锁，因此 RC 下当前读可能出现幻读。

purge 机制

- INSERT 的 undo 日志在事务提交后即可释放
- UPDATE/DELETE 的 undo 日志需要支撑 MVCC，不能立即删除
- 被 DELETE 标记的记录也不会立即物理删除（MVCC 可能还需要该版本）
- InnoDB 后台运行 purge 线程，在确认没有任何活跃 ReadView 需要访问这些旧版本后，才真正回收 undo 日志和物理删除记录

锁

并发控制与锁的分类

并发事务访问同一数据有三种情况：

- 读-读：互不影响，无需控制
- 写-写：必须排队，通过锁实现。当前事务对记录加锁后，其他事务修改同一记录时必须等待
- 读-写：可能产生脏读、不可重复读、幻读。两种解决方案：
  - MVCC（多版本并发控制）：读操作读取快照版本，写操作针对最新版本，读写不冲突，性能好
  - 加锁：读写操作排队执行，适用于必须读到最新数据的业务场景

锁按粒度分为表级锁和行级锁；按模式分为共享锁（S 锁）和排他锁（X 锁）：

- S 锁（Shared Lock）：读取记录前获取，多个事务可同时持有同一记录的 S 锁
- X 锁（Exclusive Lock）：修改记录前获取，与任何锁互斥

S/X 锁兼容性：

	S	X
S	兼容	冲突
X	冲突	冲突

表级锁

S 锁与 X 锁

InnoDB 执行普通 DML（SELECT/INSERT/UPDATE/DELETE）时不会主动加表级 S/X 锁。

DDL 语句（ALTER TABLE、DROP TABLE）通过 Server 层的元数据锁（MDL）实现表级互斥：DDL 执行时，其他会话的 DML 被阻塞；反之亦然。

手动加表锁（极少使用）：

```
LOCK TABLES t READ;   -- 加表级 S 锁
LOCK TABLES t WRITE;  -- 加表级 X 锁
```

> InnoDB 的优势在于行级锁，尽量避免使用 LOCK TABLES。

意向锁（IS/IX）

在给行加锁之前，InnoDB 会先在表级别加对应的意向锁：

- IS 锁（Intention Shared）：准备给某行加 S 锁前，先对表加 IS 锁
- IX 锁（Intention Exclusive）：准备给某行加 X 锁前，先对表加 IX 锁

意向锁的唯一目的：在后续需要加表级 S/X 锁时，能快速判断表中是否有行被锁定，避免逐行遍历。

IS 与 IX 之间完全兼容——它们只在与表级 S/X 锁交互时才有冲突。

表级锁兼容矩阵：

	X	IX	S	IS
X	冲突	冲突	冲突	冲突
IX	冲突	兼容	冲突	兼容
S	冲突	冲突	兼容	兼容
IS	冲突	兼容	兼容	兼容

AUTO-INC 锁

对含 `AUTO_INCREMENT` 列的表执行插入时，需要保证自增值的分配不冲突。MySQL 提供两种机制：

1. AUTO-INC 锁：表级锁，在整条插入语句执行期间持有（注意是语句级别，不是事务级别），其他事务的插入被阻塞直到该语句结束。适用于无法预知插入行数的语句（`INSERT ... SELECT`、`LOAD DATA`）
2. 轻量级互斥量：仅在分配自增值时短暂持有，分配完立即释放，不等语句结束。适用于能预知插入行数的简单 INSERT

通过 `innodb_autoinc_lock_mode` 控制：

值	策略	说明
0	全部用 AUTO-INC 锁	最安全，并发最差
1	混合模式（默认）	可预知行数用轻量锁，否则用 AUTO-INC 锁
2	全部用轻量锁	并发最好，但自增值可能不连续，基于 STATEMENT 的主从复制不安全

行级锁（InnoDB）

InnoDB 的行锁加在索引记录上，有以下几种类型：

Record Lock（记录锁）

锁住单条索引记录。官方类型名 `LOCK_REC_NOT_GAP`。

- S 型记录锁：其他事务可加 S 型记录锁，不可加 X 型记录锁
- X 型记录锁：其他事务既不可加 S 型也不可加 X 型记录锁

Gap Lock（间隙锁）

锁住记录前面的间隙，不锁记录本身。官方类型名 `LOCK_GAP`。

作用：防止其他事务向该间隙插入新记录，用于在 RR 隔离级别下解决幻读问题。

假设索引中有记录 3、8、15，对记录 8 加 Gap Lock，则锁住的是区间 `(3, 8)`，禁止在此间隙中插入新记录。

关键特性：

- Gap 锁之间完全兼容（共享和独占的 Gap 锁效果相同）
- Gap 锁不阻止其他事务对同一记录加 Record Lock
- 最后一条记录之后的间隙通过对页面的 Supremum 伪记录加 Gap Lock 来保护

Next-Key Lock（临键锁）

Record Lock + Gap Lock 的组合，既锁住记录本身，又锁住记录前面的间隙。官方类型名 `LOCK_ORDINARY`。

对记录 8 加 Next-Key Lock = 锁住区间 `(3, 8]`（左开右闭）。

InnoDB 在 RR 隔离级别下，行锁的默认加锁方式就是 Next-Key Lock。

Insert Intention Lock（插入意向锁）

当事务向某个间隙插入记录、但该间隙已被其他事务加了 Gap Lock 时，插入操作会阻塞，此时会生成一个插入意向锁（类型名 `LOCK_INSERT_INTENTION`），表示"等待插入"。

关键特性：

- 插入意向锁是一种特殊的 Gap Lock
- 多个事务向同一间隙的不同位置插入时互不阻塞（例如间隙 (3, 8) 中，事务 A 插入 4、事务 B 插入 5 不冲突）
- 插入意向锁不阻止任何其他类型的锁

隐式锁

INSERT 操作一般不显式加锁，通过隐式锁保护新插入的记录：

- 聚簇索引：记录的 `trx_id` 隐藏列记录了插入事务的 ID。其他事务想对该记录加锁时，发现 `trx_id` 对应事务仍活跃，就会先帮插入事务创建 X 锁，再让自己进入等待
- 二级索引：通过页面的 `PAGE_MAX_TRX_ID` 属性判断，必要时回表到聚簇索引做同样的检查

行锁兼容矩阵

已持有（行）↓ \ 请求→	Record Lock	Gap Lock	Next-Key Lock	Insert Intention
Record Lock	S-S 兼容，其余冲突	兼容	S-S 兼容，其余冲突	兼容
Gap Lock	兼容	兼容	兼容	冲突
Next-Key Lock	S-S 兼容，其余冲突	兼容	S-S 兼容，其余冲突	冲突
Insert Intention	兼容	兼容	兼容	兼容

核心规律：Gap Lock / Next-Key Lock 会阻塞 Insert Intention Lock；其余行锁类型之间，仅 X 与 X、X 与 S 的 Record Lock 部分互斥。

加锁分析

SELECT 语句的加锁

语句	加锁方式
`SELECT ... FROM`（普通查询）	不加锁，使用 MVCC 快照读（RC/RR 级别下）
`SELECT ... LOCK IN SHARE MODE`	对读取的记录加 S 型行锁（锁定读）
`SELECT ... FOR SHARE`（MySQL 8.0+）	同上，语法更规范
`SELECT ... FOR UPDATE`	对读取的记录加 X 型行锁（锁定读）

SERIALIZABLE 级别下，普通 SELECT 自动转为 `SELECT ... LOCK IN SHARE MODE`。

INSERT/UPDATE/DELETE 的加锁

- DELETE：定位记录后加 X 锁，再执行删除标记
- UPDATE：
  - 未修改键值且存储空间不变：原地加 X 锁修改
  - 未修改键值但存储空间变化：加 X 锁 → 删除原记录 → 插入新记录（隐式锁保护）
  - 修改了键值：等同于 DELETE + INSERT
- INSERT：一般不加锁（隐式锁保护），插入位置有 Gap Lock 则阻塞并生成插入意向锁

不同隔离级别下的加锁差异

隔离级别	加锁特点
READ UNCOMMITTED	写操作加 Record Lock，读操作不加锁（可脏读）
READ COMMITTED	写操作加 Record Lock，不加 Gap Lock，因此不防幻读
REPEATABLE READ	默认使用 Next-Key Lock（Record + Gap），可防止幻读
SERIALIZABLE	类似 RR，但普通 SELECT 也会加 S 锁，完全串行化

RC 级别的一个优化：查询条件不匹配的记录上的锁会立即释放，而不是等事务结束。

死锁

死锁产生条件

两个或多个事务互相持有对方等待的锁，形成循环等待。四个必要条件：

1. 互斥：锁是排他的
2. 持有并等待：事务持有已获得的锁，同时等待新锁
3. 不可剥夺：已持有的锁不能被强制释放
4. 循环等待：事务之间形成等待环路

MySQL 死锁检测与处理

InnoDB 采用等待图（wait-for graph）进行死锁检测：将事务作为节点，等待关系作为边构建有向图，图中出现环即表示死锁。检测到死锁后，选择代价最小的事务回滚（持有锁最少的），返回 `ERROR 1213 (40001): Deadlock found`。

另一种策略是等待超时（`innodb_lock_wait_timeout`，默认 50 秒），超时后事务回滚。但单靠超时响应太慢，InnoDB 默认开启主动死锁检测（`innodb_deadlock_detect = ON`）。

常见死锁场景

场景一：交叉更新

```
事务A: UPDATE t SET ... WHERE id = 1;  -- 获取 id=1 的 X 锁
事务B: UPDATE t SET ... WHERE id = 2;  -- 获取 id=2 的 X 锁
事务A: UPDATE t SET ... WHERE id = 2;  -- 等待 id=2 的 X 锁（B 持有）
事务B: UPDATE t SET ... WHERE id = 1;  -- 等待 id=1 的 X 锁（A 持有）→ 死锁
```

场景二：Gap Lock 导致的死锁

```
-- 假设表中无 id=5 的记录，id=3 和 id=8 存在
事务A: SELECT * FROM t WHERE id = 5 FOR UPDATE;  -- 对 (3,8) 加 Gap Lock
事务B: SELECT * FROM t WHERE id = 5 FOR UPDATE;  -- 也对 (3,8) 加 Gap Lock（Gap Lock 兼容）
事务A: INSERT INTO t VALUES(5, ...);  -- 被 B 的 Gap Lock 阻塞
事务B: INSERT INTO t VALUES(5, ...);  -- 被 A 的 Gap Lock 阻塞 → 死锁
```

避免死锁的建议

1. 按固定顺序访问表和行（最有效的手段）
2. 事务尽量小，减少持锁时间
3. 合理使用索引，避免全表扫描导致锁住过多行
4. 在 RC 隔离级别下，Gap Lock 被禁用，可减少死锁概率（需权衡幻读风险）
5. 使用 `SHOW ENGINE INNODB STATUS\G` 查看最近一次死锁信息进行分析

实战：常见加锁场景

以下分析基于 RR 隔离级别，使用主键索引（唯一索引类似）。

等值查询命中

```
SELECT * FROM t WHERE id = 8 FOR UPDATE;
```

记录存在时：对 id=8 加 Record Lock（X）——Next-Key Lock 退化，不需要 Gap Lock。

等值查询未命中

```
SELECT * FROM t WHERE id = 5 FOR UPDATE;
```

记录不存在（id 在 3 和 8 之间）：对 id=8 加 Gap Lock，锁住 `(3, 8)` 间隙，防止幻影插入。Next-Key Lock 退化为纯 Gap Lock。

范围查询

```
SELECT * FROM t WHERE id >= 3 AND id < 8 FOR UPDATE;
```

- id=3 命中：加 Record Lock
- 继续向右扫描至 id=8（不满足 `< 8`）：对 id=8 加 Gap Lock 或 Next-Key Lock（取决于 MySQL 版本优化）
- 锁住区间 `[3, 8)` 的记录和间隙

唯一索引 vs 普通索引

场景	唯一索引	普通索引
等值查询命中	Record Lock（退化）	Next-Key Lock + 右侧 Gap Lock
等值查询未命中	Gap Lock（退化）	Gap Lock（退化）

唯一索引由于唯一性约束，等值命中时确定只有一条，无需 Gap Lock。普通索引可能有重复值，需要额外的 Gap Lock 防止幻读。

加锁范围速查（主键 1，3，8，15，20）

查询条件	加锁范围
`id = 8`	Record Lock on 8
`id = 5`（不存在）	Gap Lock (3, 8)
`id >= 8 AND id < 15`	Next-Key Lock (3, 8] + Gap Lock (8, 15)
`id > 18`	Next-Key Lock (15, 20] + Next-Key Lock (20, +∞)